Apostila 1 — Fundamentos de Python para Economia (Teoria Completa)

Teoria, exemplos executados e boas práticas, com foco em economia

Sumário

Apostila 1 — Fundamentos de Python para Economia (Teoria Completa) 1
- Como Usar Esta Apostila 4
- Capítulo 1 — Por que Python 4
  - 1.1. Um pouco de história 4
  - 1.2. Por que economistas migraram para Python 4
  - 1.3. O que você vai construir com isso 5
- Capítulo 2 — Como o Python executa seu código 6
  - 2.1. Linguagem interpretada vs. compilada 6
  - 2.2. Três formas de rodar Python 6
  - 2.3. `print()` — sua ferramenta de diagnóstico mais usada 7
- Capítulo 3 — Variáveis e o modelo de memória 8
  - 3.1. O que realmente é uma variável 8
  - 3.2. Regras e convenções de nomenclatura 8
  - 3.3. Tipagem dinâmica 9
  - 3.4. Exemplo Resolvido — Rastreando rótulos 9
- Capítulo 4 — Tipos de dados primitivos 10
  - 4.1. Números inteiros (`int`) 10
  - 4.2. Números decimais (`float`) e um alerta importante 10
  - 4.3. Texto (`str`) 11
  - 4.4. Booleano (`bool`) 11
  - 4.5. Verificando e convertendo tipos 12
  - 4.6. Exemplo Resolvido — Convertendo valor monetário em formato brasileiro 12
- Capítulo 5 — Operadores 14
  - 5.1. Aritméticos 14
  - 5.2. Comparação (resultado é sempre `bool`) 14
  - 5.3. Lógicos e curto-circuito 14
  - 5.4. Exemplo Resolvido — Juro real exato (Fisher) vs. aproximado 15
- Capítulo 6 — Strings em detalhe 16
  - 6.1. Strings são sequências imutáveis 16
  - 6.2. Concatenação e f-strings 17
  - 6.3. Mini-linguagem de formatação 17
  - 6.4. Métodos úteis de string 17
  - 6.6. Exemplo Resolvido — Relatório formatado com alinhamento 18
- Capítulo 7 — Listas 19
  - 7.1. O que é uma lista 19
  - 7.2. Indexação e fatiamento 19
  - 7.3. Mutabilidade e o problema da aliasing 19
  - 7.4. Métodos de lista 20
  - 7.6. Exemplo Resolvido — Corrigindo outliers sem afetar a lista original 20
- Capítulo 8 — Dicionários 22
  - 8.1. O que é um dicionário e por que ele é rápido 22
  - 8.2. Acessando e modificando 22
  - 8.3. Métodos e iteração 23
  - 8.4. Dicionários aninhados 23
  - 8.5. Lista vs. dicionário — quando usar qual 23
  - 8.6. Exemplo Resolvido — Combinando dois dicionários 24
- Capítulo 9 — Condicionais 25
  - 9.1. if / elif / else 25

9.2. Truthiness — o que conta como “verdadeiro” 25
9.3. Operador ternário 26
9.4. Combinando condições 26
9.5. Exemplo Resolvido — Tratando dado ausente antes de comparar 26
Capítulo 10 — Laços de repetição 27
10.1. `for` — percorrendo sequências (iteráveis) 27
10.2. `while` — repetir enquanto uma condição for verdadeira 28
10.3. `break` e `continue` 29
10.4. Nota sobre desempenho: laços vs. vetorização 29
10.5. Exemplo Resolvido — Trajetória de patrimônio mês a mês 29
Capítulo 11 — Funções 31
11.1. Por que funções importam além de “reaproveitar código” 31
11.2. Função sem parâmetros e sem retorno 31
11.3. Parâmetros e retorno 31
11.4. Vários parâmetros 31
11.5. Parâmetros com valor padrão — e uma armadilha a evitar 32
11.6. Retornando múltiplos valores 32
11.7. Escopo: variáveis locais vs. globais 33
11.8. Exemplo Resolvido — Função pura vs. função com efeito colateral 33
Capítulo 12 — List comprehension 34
12.1. Sintaxe e motivação 34
12.2. Quando (não) usar 35
12.3. Exemplo Resolvido — Achatando um dicionário aninhado 35
Capítulo 13 — Erros e depuração 36
13.1. Lendo uma mensagem de erro 36
13.2. Tratando erros com try/except 37
13.3. Exemplo Resolvido — Tratando dois tipos de erro diferentes 37
Capítulo 14 — Boas práticas de código 39
Capítulo 15 — Exercícios para Executar (na mão) 40
Exercício 1 — Trace de execução 40
Exercício 2 — Juros compostos na mão 40
Exercício 3 — Avaliação lógica 40
Exercício 4 — Encontre o erro 40
Exercício 5 — Pseudocódigo de decisão 41
Capítulo 16 — Exercícios para Executar (em código) 41
Exercício 1 — Inflação acumulada 41
Exercício 2 — Maior e menor PIB 41
Exercício 3 — Simulação de juros compostos mês a mês 41
Exercício 4 — Valores acima da média 41
Exercício 5 — Média geométrica do IPCA 41
Exercício 6 — Meses em recessão 41
Exercício 7 — Crescimento a partir de dicionário aninhado 41
Exercício 8 — Entrada validada com try/except 41
Exercício 9 — Classificador de cenário macroeconômico 42
Exercício 10 — Séries “lado a lado” com list comprehension 42
Exercício 11 — Limpando dados com taxas inválidas 42
Exercício 12 — Fechamentos de trimestre 42
Exercício 13 — Parseando um rótulo de período 42
Exercício 14 — Semestres via fatiamento 42
Exercício 15 — Cópia segura de uma base 42
Exercício 16 — Contador de cenários 42
Exercício 17 — Estatísticas sem bibliotecas 42
Exercício 18 — Meses até dobrar (busca com while) 42

Exercício 19 — Acesso seguro em dicionário aninhado 43
Exercício 20 — Projeto integrador: painel mensal 43
Capítulo 17 — Resumo 43
Capítulo 18 — Mini-projeto Resolvido: Lendo e Analisando uma Série do BCB 43
18.1. Ler e converter 43
18.2. Analisar 44
18.3. Relatório formatado e tratamento de erro 44
Capítulo 19 — Erros Comuns e Teste Rápido 45
19.1. Os erros mais comuns de quem está começando 45
19.2. Teste rápido: verdadeiro ou falso (com gabarito) 45
Próxima apostila 46

Como Usar Esta Apostila

Esta é a primeira de uma série de apostilas de Python para economia. Ela ensina a **linguagem em si**: como o Python pensa (memória, tipos, sequências, funções, erros). Sem esse alicerce, as bibliotecas de análise de dados parecem mágica — e a mágica falha de formas misteriosas. Com ele, os erros ficam legíveis.

Cada tópico segue três camadas: **(1) o porquê** (o que acontece "por baixo"), **(2) o exemplo** (código curto, *executado* — os resultados impressos foram obtidos rodando o código) e **(3) a prática** (exercícios no fim, sem gabarito, para você resolver). Caixas coloridas:

Exemplo resolvido — modelo
Um exemplo executado e comentado.

Intuição
A ideia por trás do mecanismo, sem jargão.

Aprofundando a teoria — modelo
O que acontece "por baixo": modelo mental preciso.

Atenção — erro comum
Um erro clássico de iniciante (e de veterano distraído).

No mercado brasileiro
Como o conceito aparece no trabalho de um economista.

Como estudar. (1) Digite o código (não copie e cole). (2) Antes de rodar, preveja a saída. (3) Quando errar a previsão, descubra *por quê*: é aí que o modelo mental se corrige. (4) Só então olhe a explicação.

Capítulo 1 — Por que Python

1.1. Um pouco de história

Python foi criado por Guido van Rossum e lançado em 1991. O nome não vem da cobra, mas do grupo de comédia britânico *Monty Python's Flying Circus* — van Rossum queria uma linguagem "divertida de usar". Essa origem despretensiosa explica um traço central da linguagem: ela foi desenhada para ser lida por humanos primeiro, e executada por máquinas depois.

Isso está formalizado no **Zen of Python**, um conjunto de 19 princípios que qualquer instalação de Python exibe ao rodar `import this`. Dois deles resumem o espírito da linguagem:

> *"Readability counts."* (Legibilidade importa) *"There should be one — and preferably only one — obvious way to do it."* (Deve haver uma — e de preferência só uma — forma óbvia de fazer algo)

Para um economista, isso importa mais do que parece: um script de análise de dados frequentemente é lido meses depois — por você mesmo, por um coautor, por um parecerista — e precisa ser compreensível sem arqueologia de código.

1.2. Por que economistas migraram para Python

Até os anos 2010, o ferramental padrão de um economista era Excel para planilhas, Stata ou EViews para econometria, e talvez MATLAB para modelos computacionais. Cada uma dessas ferramentas resolve bem um problema específico, mas nenhuma resolve todos:

Ferramenta	Ponto forte	Limitação
Excel	Visual, fácil de começar	Não escala (arquivos grandes travam), difícil de auditar, erros silenciosos em fórmulas
Stata/EViews	Comandos econométricos prontos	Licença paga, menos flexível para dados não tabulares, comunidade menor de desenvolvimento
MATLAB	Forte em álgebra linear/otimização	Licença cara, pouco usado fora de engenharia/finanças quantitativas
R	Excelente para estatística	Sintaxe menos amigável para programação geral, ecossistema mais fragmentado
Python	Faz tudo isso e mais (dados, gráficos, econometria, machine learning, web, automação)	Curva de aprendizado inicial um pouco maior que Excel

Três fatores concretos explicam a adoção maciça de Python em bancos centrais (BCB, Fed, BCE), organismos internacionais (FMI, Banco Mundial, OCDE) e departamentos de economia:

1. **Gratuidade e abertura.** Python é open source. Um pesquisador em qualquer lugar do mundo pode rodar exatamente o mesmo código sem pagar licença — o que também torna a pesquisa **reprodutível**: outro economista pode reexecutar sua análise linha por linha.
2. **Ecossistema unificado.** As mesmas bibliotecas (`pandas`, `numpy`, `statsmodels`, `scikit-learn`) cobrem desde a limpeza de uma planilha até a estimação de um VAR ou o treino de um modelo preditivo. Você não precisa trocar de ferramenta a cada etapa do trabalho.
3. **Automação.** Um script Python pode ser reexecutado automaticamente — todo mês, ao sair um novo IPCA, por exemplo — sem intervenção manual. Isso é o que torna possível um dashboard macroeconômico que se atualiza sozinho.

1.3. O que você vai construir com isso

Ao final desta série de apostilas, o objetivo é que você consiga: baixar dados econômicos reais (BCB, IBGE, FMI), limpá-los e organizá-los, visualizá-los de forma profissional, rodar regressões e modelos de séries temporais, e empacotar tudo isso em projetos que podem virar um TCC, um relatório de research ou um portfólio.

Tudo começa, porém, na base: entender como o Python pensa. Esse é o assunto desta apostila.

Ferramenta	Ponto forte	Limite	Quando usar
Excel / Planilhas	Visual, imediato, universal	Difícil de auditar e de repetir em escala	Cálculo pontual, comunicação com não técnicos
Stata	Econometria padrão em artigos acadêmicos	Licença paga; linguagem de propósito específico	Replicar artigos, equipes acostumadas
R	Estatística e gráficos; ótima comunidade acadêmica	Menos usado fora da academia e estatística	Estatística aplicada, pesquisa
Python	Linguagem geral: dados, automação, web, IA	Menos "pronto" para estatística que o R	Pipeline completo, automação, produção

> **Aprofundando a teoria — o ecossistema que esta série percorre**
> Python sozinho faz pouco em análise de dados; o poder vem de **bibliotecas** construídas sobre ele:
>
> - **NumPy** — arrays numéricos rápidos (a base de tudo).
> - **pandas** — tabelas (DataFrames), séries temporais, leitura de arquivos (Apostila 2).
> - **Matplotlib / seaborn** — gráficos (Apostila 3).
> - **statsmodels / scikit-learn** — econometria e aprendizado de máquina (Apostila 4).

- **python-bcb, wbdata, yfinance** — acesso a dados do Banco Central, Banco Mundial, mercado.

Todas dependem dos fundamentos desta apostila: listas viram arrays, dicionários viram DataFrames, funções viram transformações, erros viram sinais de dado sujo.

Capítulo 2 — Como o Python executa seu código

2.1. Linguagem interpretada vs. compilada

Linguagens como C ou Rust são **compiladas**: existe uma etapa prévia em que todo o código-fonte é traduzido de uma vez para instruções de máquina, gerando um executável. Só depois disso o programa roda.

Python é **interpretado**: um programa chamado interpretador lê seu código linha por linha (mais precisamente, instrução por instrução) e vai executando cada uma imediatamente, sem gerar um executável separado. Isso tem duas consequências práticas:

- **Feedback mais rápido.** Você escreve uma linha, roda, vê o resultado — sem esperar uma compilação. É por isso que Python é tão usado para exploração de dados: o ciclo "tentei algo → vi o resultado → ajustei" é curtíssimo.
- **Erros aparecem só quando a linha é executada.** Um script pode ter um erro de sintaxe na linha 200 e ainda assim rodar perfeitamente até chegar lá, porque o interpretador não analisa o arquivo inteiro de antemão da mesma forma que um compilador analisaria.

Tecnicamente, o interpretador CPython (a implementação padrão de Python) primeiro traduz seu código para uma representação intermediária chamada **bytecode**, e depois executa esse bytecode numa máquina virtual. Isso não muda o comportamento do ponto de vista prático, mas explica por que arquivos `.pyc` aparecem em pastas `__pycache__` — são o bytecode cacheado.

2.2. Três formas de rodar Python

Ambiente	Quando usar
Script (`arquivo.py`)	Análises que você quer repetir, versionar e automatizar
REPL (terminal interativo, `python`)	Testar uma linha rapidamente, sem salvar nada
Notebook (Jupyter)	Exploração de dados, onde você quer ver gráficos e tabelas ao lado do código, célula por célula

Um arquivo `.py` é executado de cima para baixo, instrução por instrução:

```
# isso é um comentário — o computador ignora
# serve para explicar o código, não para ser executado

print("Hello, Economia!")  # print exibe algo na tela
```

```
Hello, Economia!
```

Salve como `teste.py` e execute pelo terminal:

```
python teste.py
```

Saída:

```
Hello, Economia!
```

2.3. `print()` — sua ferramenta de diagnóstico mais usada

`print()` exibe qualquer valor na tela. Antes de existirem debuggers sofisticados, é a forma mais rápida (e ainda a mais usada no dia a dia) de entender o que um programa está fazendo em cada etapa.

```
print("IPCA de junho: 0.16%")

# print aceita múltiplos valores, separados por vírgula
mes = "junho"
valor = 0.16
print("IPCA de", mes, ":", valor, "%")
```

```
IPCA de junho: 0.16%
IPCA de junho : 0.16 %
```

Um hábito recomendado: sempre que um resultado não for o esperado, adicione `print()` antes e depois da linha suspeita para ver os valores intermediários. Isso é chamado, informalmente, de "debugging por print", e resolve a maioria dos problemas de um iniciante.

Aprofundando a teoria — do texto ao resultado: as etapas de execução

Quando você roda um arquivo `.py`, o interpretador faz, em ordem: (1) **análise léxica**, quebrando o texto em "palavras" (tokens); (2) **análise sintática**, organizando-as numa árvore (AST) que reflete a estrutura do programa — aqui nascem os `SyntaxError`; (3) **compilação para bytecode**, instruções de uma máquina virtual; (4) **execução** do bytecode, linha a linha — aqui nascem `NameError`, `TypeError`, `ZeroDivisionError` e demais erros de *execução*. Um programa pode ter 100 linhas corretas e uma 101ª com erro de sintaxe: **nenhuma** linha roda, porque a análise sintática falha antes. Já um erro de execução só aparece quando o código chega naquela linha.

```
import ast
arvore = ast.parse("juros = capital * taxa")
print(ast.dump(arvore, indent=2))
```

```
Module(
  body=[
    Assign(
      targets=[
        Name(id='juros', ctx=Store())],
      value=BinOp(
        left=Name(id='capital', ctx=Load()),
        op=Mult(),
        right=Name(id='taxa', ctx=Load()))),
  type_ignores=[])
```

Exemplo resolvido — erro de sintaxe e erro de execução

O código abaixo tem um erro de *sintaxe* (parêntese aberto) e outro de *execução*. Compare quando cada um é detectado:

```
codigo_sintaxe = "print('antes')\nx = (1 + 2\nprint('depois')"
try:
    compile(codigo_sintaxe, "<exemplo>", "exec")
except SyntaxError as erro:
    print(f"Erro de sintaxe detectado ANTES de rodar qualquer linha: {erro.msg} (linha
{erro.lineno})")

codigo_execucao = "print('antes')\nx = 1 / 0\nprint('depois')"
try:
```

```
    exec(compile(codigo_execucao, "<exemplo>", "exec"))
except ZeroDivisionError as erro:
    print(f"Erro de execução: {erro} (a primeira linha já havia rodado)")
```

```
Erro de sintaxe detectado ANTES de rodar qualquer linha: '(' was never closed (linha 2)
antes
Erro de execução: division by zero (a primeira linha já havia rodado)
```

No mercado brasileiro
Reprodutibilidade. Em pesquisa e em trabalho profissional, "funciona no meu computador" não basta. Um ambiente isolado (`venv`) e um arquivo de dependências (`requirements.txt`) registram as bibliotecas e versões usadas, para que o resultado possa ser reproduzido meses depois ou por um colega:

```
python -m venv .venv                # cria um ambiente isolado
.venv\Scripts\activate              # ativa (Windows); no Linux/macOS: source .venv/
bin/activate
pip install pandas numpy matplotlib  # instala dentro do ambiente
pip freeze > requirements.txt       # registra as versões exatas
```

Capítulo 3 — Variáveis e o modelo de memória

3.1. O que realmente é uma variável

Em muitas linguagens (C, por exemplo), uma variável é uma **caixa** com um espaço fixo de memória, e atribuir um valor é colocar algo dentro dessa caixa. Em Python, o modelo é diferente e vale a pena entender desde já, porque evita confusões mais adiante (especialmente com listas, no Capítulo 7).

Em Python, uma variável é um **nome (rótulo)** que aponta para um objeto na memória. Atribuir um valor não coloca nada "dentro" da variável — cria um objeto em algum lugar da memória e faz o nome apontar para ele.

```
ipca_junho = 0.16
selic = 10.50
pib_2025 = 3.2

print(ipca_junho)   # 0.16
print(selic)        # 10.5
```

```
0.16
10.5
```

Aqui, `ipca_junho` não é uma caixa contendo `0.16` — é um rótulo colado no objeto `0.16`, que existe na memória do computador. Essa distinção parece sutil agora, mas explica comportamentos que confundem iniciantes, como o fato de duas variáveis poderem apontar para o **mesmo** objeto (voltaremos a isso com listas).

3.2. Regras e convenções de nomenclatura

Regras (o Python exige, senão dá erro): - Pode conter letras, números e underscore `_` - Não pode começar com número - Não pode ser uma palavra reservada (`if`, `for`, `class`, etc.) - Maiúsculas e minúsculas são diferentes (`IPCA` ≠ `ipca`)

Convenções (o Python não exige, mas a comunidade segue — o guia oficial de estilo chama-se **PEP 8**): - Nomes de variáveis em `snake_case`: `taxa_juros`, não `taxaJuros` nem `TaxaJuros` - Nomes claros em vez de abreviações obscuras: `taxa_juros` é melhor que `tj` - Constantes (valores que não mudam) em maiúsculas: `TAXA_SELIC_META = 10.50`

Seguir PEP 8 não é purismo estético: em economia, código é frequentemente compartilhado (com coautores, orientadores, revisores), e nomes claros substituem metade da documentação que você precisaria escrever.

3.3. Tipagem dinâmica

Python é **dinamicamente tipado**: uma variável não tem um tipo fixo — o tipo pertence ao objeto ao qual ela aponta no momento, e pode mudar:

```
x = 10        # x aponta para um int
print(type(x))  # <class 'int'>

x = "dez"     # agora x aponta para um str
print(type(x))  # <class 'str'>
```

```
<class 'int'>
<class 'str'>
```

Isso contrasta com linguagens **estaticamente tipadas** (C, Java), onde uma variável declarada como inteiro só pode guardar inteiros para sempre. A tipagem dinâmica de Python dá flexibilidade e reduz a verbosidade do código, mas exige disciplina: como o interpretador não vai barrar você antecipadamente, erros de tipo (somar texto com número, por exemplo) só aparecem quando a linha problemática é executada — não antes.

3.4. Exemplo Resolvido — Rastreando rótulos

Enunciado: sem rodar no computador, preveja o que o código abaixo imprime e explique o porquê usando o modelo de "rótulo" (não "caixa") apresentado neste capítulo.

```
selic = 10.5
selic_copia = selic
selic = 11.0
print(selic_copia)
```

```
10.5
```

Solução comentada:

A saída é `10.5`.

1. `selic = 10.5` cria o objeto `10.5` na memória e faz o rótulo `selic` apontar para ele.
2. `selic_copia = selic` **não copia o número** — cria um segundo rótulo, `selic_copia`, apontando para o **mesmo objeto** `10.5` que `selic` já aponta. Neste momento, os dois rótulos apontam para o mesmo lugar.
3. `selic = 11.0` cria um objeto **novo**, `11.0`, e faz o rótulo `selic` passar a apontar para ele. O objeto antigo (`10.5`) continua existindo na memória, e `selic_copia` continua apontando para ele — porque reatribuir `selic` só move o rótulo `selic`, não afeta nenhum outro rótulo.
4. Por isso `print(selic_copia)` ainda mostra `10.5`.

Isso funciona de forma "segura" porque números (`int`, `float`) são **imutáveis**: reatribuir sempre cria um objeto novo, nunca modifica o objeto existente. No Capítulo 7 (Listas), veremos que esse raciocínio muda quando o objeto é **mutável** — lá, alterar através de um rótulo *afeta* o outro rótulo que aponta para o mesmo objeto.

> **Aprofundando a teoria — variáveis são rótulos: identidade, igualdade e contagem de referências**
> Em `x = [1, 2]`, o Python cria o objeto lista na memória e **amarra o nome** `x` a ele. `y = x` não copia nada: cria *outro rótulo* para o mesmo objeto. Duas perguntas diferentes:
>
> - `a == b` — os **valores** são iguais?
> - `a is b` — são o **mesmo objeto** (mesma identidade)?

Para tipos imutáveis (números, strings, tuplas), o aliasing é inofensivo, porque o objeto não muda. Para tipos **mutáveis** (listas, dicionários), duas variáveis para o mesmo objeto significam que alterar por uma delas altera "as duas" — fonte clássica de bugs.

```
a = [1, 2, 3]
b = a                  # outro rótulo para o MESMO objeto
c = list(a)            # cópia: novo objeto com os mesmos valores
b.append(4)
print(f"a = {a} | b = {b} | c = {c}")
print(f"a is b: {a is b} | a is c: {a is c} | a == c (valores): {a == [1, 2, 3, 4]} / c ==
[1, 2, 3]: {c == [1, 2, 3]}")

x, y = 256, 256
m, n = 10_000, 10_000
print(f"Inteiros pequenos (256) são reaproveitados; para 10_000 isso é um detalhe de
implementação: x == y -> {x == y}")
```

```
a = [1, 2, 3, 4] | b = [1, 2, 3, 4] | c = [1, 2, 3]
a is b: True | a is c: False | a == c (valores): True / c == [1, 2, 3]: True
Inteiros pequenos (256) são reaproveitados; para 10_000 isso é um detalhe de implementação:
x == y -> True
```

Atenção — erro comum
Use `==` para comparar *valores* e `is` apenas para comparar com `None` (`if x is None`). O comportamento de `is` com números e strings depende de otimizações da implementação, e não deve ser usado para testar igualdade.

Capítulo 4 — Tipos de dados primitivos

4.1. Números inteiros (`int`)

```
ano = 2026
populacao = 214_000_000  # underscore é só separador visual, ignorado pelo Python
```

Diferentemente de linguagens como C, o `int` de Python não tem limite de tamanho fixo (32 ou 64 bits) — ele cresce conforme necessário, limitado apenas pela memória do computador. Isso significa que, em Python, você pode multiplicar números arbitrariamente grandes sem *overflow*.

4.2. Números decimais (`float`) e um alerta importante

```
ipca = 0.16
cambio = 5.45
pib_per_capita = 45000.00
```

Ponto flutuante não é exato. Computadores representam números decimais em base binária, e frações decimais simples (como 0,1) não têm representação binária finita — da mesma forma que 1/3 não tem representação decimal finita. O resultado é um pequeno erro de arredondamento:

```
print(0.1 + 0.2)        # 0.30000000000000004 — não é 0.3!
print(0.1 + 0.2 == 0.3) # False
```

```
0.30000000000000004
False
```

Isso **não é um bug do Python** — é uma característica de como todo computador representa ponto flutuante (IEEE 754), presente em praticamente qualquer linguagem.

Por que isso acontece, exatamente. Um computador guarda um `float` em base **binária** (potências de 2), não em base decimal (potências de 10). Em base 10, uma fração como $1/3$ não tem representação finita ($0{,}333\ldots$); em base 2, o mesmo problema ocorre com frações que são triviais em base 10, como $0{,}1$. Expandindo $0{,}1$ como soma de potências de 2:

$$0{,}1 = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \ldots = \sum_{i=1}^{\infty} a_i \cdot 2^{-i}$$

essa série **nunca fecha exatamente** — é uma dízima binária periódica, análoga a $\frac{1}{3} = 0{,}333\ldots$ em base 10. O computador guarda apenas um número finito de bits (52 bits de mantissa em `IEEE 754` double precision), então armazena a **melhor aproximação possível** de $0{,}1$, não o valor exato. Ao somar duas aproximações (`0.1` e `0.2`), os erros de arredondamento — cada um da ordem de 10^{-17} — se acumulam e aparecem no resultado. Você pode ver a representação binária real com:

```
from decimal import Decimal
print(Decimal(0.1))  # 0.1000000000000000055511151231257827021181583404541015625
```

```
0.1000000000000000055511151231257827021181583404541015625
```

Ou seja: `0.1` do Python **já nasce** ligeiramente diferente de $\frac{1}{10}$ exato — a soma só torna esse erro, que já existia antes, visível.

Para um economista, a implicação prática é: nunca compare valores monetários ou taxas com == diretamente; arredonde antes, ou compare a diferença absoluta com uma tolerância pequena:

```
tolerancia = 1e-9
print(abs((0.1 + 0.2) - 0.3) < tolerancia)  # True
```

```
True
```

4.3. Texto (`str`)

```
pais = "Brasil"
sigla = 'BR'   # aspas simples ou duplas — tanto faz, desde que combinem no início e fim
mensagem = "O PIB cresceu 3.2% em 2025"
```

Strings em Python são sequências de caracteres Unicode, o que significa suporte nativo a acentos e caracteres especiais (`ã`, `ç`, `€`) sem configuração extra — relevante ao trabalhar com nomes de países, cidades ou variáveis em português.

4.4. Booleano (`bool`)

```
em_recessao = False
crescendo = True
```

Um detalhe técnico útil: em Python, `bool` é tecnicamente uma subclasse de `int`, onde `True` equivale a `1` e `False` equivale a `0`. Isso permite somar booleanos diretamente — útil, por exemplo, para contar quantos meses uma condição foi verdadeira:

```
meses_em_alta = [True, True, False, True]
print(sum(meses_em_alta))  # 3 — soma True como 1 e False como 0
```

```
3
```

4.5. Verificando e convertendo tipos

```
print(type(ipca))        # <class 'float'>
print(type(pais))        # <class 'str'>
print(type(em_recessao)) # <class 'bool'>

# Conversão explícita (casting)
print(int("2026"))       # 2026 (str -> int)
print(float("5.45"))     # 5.45 (str -> float)
print(str(0.16))         # "0.16" (float -> str)
print(int(5.9))          # 5 (float -> int trunca, não arredonda!)
```

```
<class 'float'>
<class 'str'>
<class 'bool'>
2026
5.45
0.16
5
```

Atenção: `int()` sobre um float **trunca** (descarta a parte decimal), não arredonda. `int(5.9)` dá `5`, não `6`. Para arredondar de verdade, use `round(5.9)`.

4.6. Exemplo Resolvido — Convertendo valor monetário em formato brasileiro

Enunciado: valores monetários brasileiros costumam vir formatados como `"R$ 1.234,56"` (ponto para milhar, vírgula para decimal) — o oposto do que `float()` entende. Escreva uma função `valor_brl_para_float` que converta essa string para um `float` utilizável em cálculos.

```
texto = "R$ 1.234,56"
```

Solução comentada:

```
def valor_brl_para_float(texto):
    limpo = texto.replace("R$", "").strip()  # remove o prefixo e espaços: "1.234,56"
    limpo = limpo.replace(".", "")            # remove separador de milhar: "1234,56"
    limpo = limpo.replace(",", ".")           # troca vírgula decimal por ponto: "1234.56"
    return float(limpo)                       # agora float() entende: 1234.56

print(valor_brl_para_float("R$ 1.234,56"))   # 1234.56
print(valor_brl_para_float("R$ 45,00"))      # 45.0
```

```
1234.56
45.0
```

O raciocínio é sempre o mesmo ao "limpar" um dado antes de convertê-lo: primeiro remover o que **não é número** (símbolo de moeda, espaços), depois ajustar os separadores para o formato que Python reconhece (ponto decimal), e só então chamar `float()`. Essa sequência — limpar, ajustar, converter — reaparece o tempo todo ao importar planilhas de fontes brasileiras na Apostila 2.

Aprofundando a teoria — dinheiro em ponto flutuante: Decimal, arredondamento e tolerância
O `float` é binário e aproximado (seção 4.2). Para **valores monetários** com contabilidade exata, existe o módulo `decimal`, que opera em base 10. E o **arredondamento** também tem armadilhas: `round()` no Python usa o **arredondamento bancário** (para o par mais próximo no empate), e o `float` armazena `2.675` como algo ligeiramente abaixo de 2,675.

```
from decimal import Decimal, ROUND_HALF_UP
import math

acumulado_float, acumulado_decimal = 0.0, Decimal("0")
for _ in range(10):                      # laço explícito (o sum() do Python 3.12 já
compensa parte do erro)
    acumulado_float += 0.1
    acumulado_decimal += Decimal("0.1")
print("Dez vezes +0,10 em float  :", acumulado_float, "| == 1.0?", acumulado_float == 1.0)
print("Dez vezes +0,10 em Decimal:", acumulado_decimal, "| == 1?", acumulado_decimal == 1)

print("\nround(0.5), round(1.5), round(2.5):", round(0.5), round(1.5), round(2.5),
"(empates vão para o par)")
print("round(2.675, 2) =", round(2.675, 2), "(esperaria 2.68)")
print("Decimal('2.675') arredondado: ", Decimal("2.675").quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP))

print("\nmath.isclose(0.1 + 0.2, 0.3):", math.isclose(0.1 + 0.2, 0.3))
```

```
Dez vezes +0,10 em float  : 0.9999999999999999 | == 1.0? False
Dez vezes +0,10 em Decimal: 1.0 | == 1? True

round(0.5), round(1.5), round(2.5): 0 2 2 (empates vão para o par)
round(2.675, 2) = 2.67 (esperaria 2.68)
Decimal('2.675') arredondado:  2.68

math.isclose(0.1 + 0.2, 0.3): True
```

No mercado brasileiro
Regra prática. Para análise econômica (taxas, índices, séries), `float` basta: o erro de 10^{-16} é irrelevante frente à incerteza dos dados. Para **contabilidade, faturamento e conciliação bancária**, onde os centavos precisam fechar, use `Decimal` (ou inteiros em centavos). E, em qualquer caso, **compare floats com tolerância**, nunca com `==`.

Aprofundando a teoria — inteiros de precisão arbitrária e o cuidado com conversões
O `int` do Python não tem limite de tamanho: `2 ** 200` é calculado exatamente. Isso é útil, por exemplo, em combinatória e em contagens enormes, mas **converter** para `float` pode perder dígitos: `float(10 ** 30 + 1) == float(10 ** 30)` é `True`.

```
print(2 ** 100)
print("float(10**30 + 1) == float(10**30) ->", float(10 ** 30 + 1) == float(10 ** 30), "(o
'+1' sumiu na conversão)")
print("int(-5.9) =", int(-5.9), "| math.floor(-5.9) =", math.floor(-5.9), "| round(-5.9)
=", round(-5.9))
```

```
1267650600228229401496703205376
float(10**30 + 1) == float(10**30) -> True (o '+1' sumiu na conversão)
int(-5.9) = -5 | math.floor(-5.9) = -6 | round(-5.9) = -6
```

Capítulo 5 — Operadores

5.1. Aritméticos

```
soma = 5 + 3           # 8
subtracao = 5 - 3      # 2
multiplicacao = 5 * 3  # 15
divisao = 5 / 3        # 1.666... (sempre retorna float)
divisao_inteira = 5 // 3  # 1 (descarta o resto)
resto = 5 % 3          # 2 (resto da divisão — o operador módulo)
potencia = 5 ** 3      # 125

# Aplicação econômica: juros compostos
valor_futuro = 1000 * (1 + 0.10) ** 5  # 1610.51
print(f"R$ {valor_futuro:.2f}")
```

```
R$ 1610.51
```

O operador módulo (`%`) parece pouco útil à primeira vista, mas aparece com frequência em análise de séries temporais — por exemplo, para descobrir se um mês é o final de trimestre (`mes % 3 == 0`).

A fórmula de juros compostos usada acima será formalizada com rigor na Apostila 6 (Matemática Financeira):

$$\text{VF} = \text{VP} \cdot (1 + i)^n$$

onde VF é o valor futuro, VP o valor presente, i a taxa de juros por período e n o número de períodos.

5.2. Comparação (resultado é sempre `bool`)

```
10 > 5    # True
10 < 5    # False
10 == 10  # True  (igualdade — repare que são dois sinais de igual)
10 != 5   # True  (diferente)
10 >= 10  # True
5 <= 3    # False
```

Um erro comum de iniciante é usar `=` (atribuição) quando o objetivo era `==` (comparação). `x = 10` **atribui** 10 a `x`; `x == 10` **pergunta** se `x` vale 10.

Python também permite **encadear comparações**, algo que a maioria das linguagens não faz:

```
selic = 10.5
print(0 < selic < 15)  # True — equivalente a (0 < selic) and (selic < 15)
```

```
True
```

5.3. Lógicos e curto-circuito

```
True and False  # False (os dois precisam ser True)
True or False   # True  (pelo menos um True)
not True        # False (inverte)
```

`and`/`or` em Python fazem **avaliação de curto-circuito**: em `a and b`, se `a` já for `False`, Python nem avalia `b` (o resultado já está decidido). Isso é usado, por exemplo, para evitar erros:

```python
lista_vazia = []

# Sem curto-circuito, acessar lista_vazia[0] daria erro (IndexError)
if len(lista_vazia) > 0 and lista_vazia[0] > 10:
    print("primeiro elemento é grande")
# como len(lista_vazia) > 0 é False, Python nunca tenta avaliar lista_vazia[0]
```

Aplicação econômica:

```python
inflacao = 0.60
desemprego = 8.5

if inflacao > 0.5 and desemprego > 8.0:
    print("Estagflação?")
```

```
Estagflação?
```

5.4. Exemplo Resolvido — Juro real exato (Fisher) vs. aproximado

Enunciado: no Capítulo 5.1 usamos `selic - ipca` como aproximação do juro real (equação de Fisher **aproximada**). A fórmula **exata** é:

$$i_{\text{real}} = \frac{1 + i_{\text{nominal}}}{1 + \text{inflação}} - 1$$

Escreva uma função `juro_real_exato(selic, ipca)` (ambos em % ao ano) e compare o resultado com a aproximação simples para `selic = 15.0` e `ipca = 10.0`. A diferença é grande ou pequena?

Solução comentada:

```python
def juro_real_exato(selic, ipca):
    i_nominal = selic / 100
    inflacao = ipca / 100
    i_real = (1 + i_nominal) / (1 + inflacao) - 1
    return i_real * 100

def juro_real_aproximado(selic, ipca):
    return selic - ipca

selic, ipca = 15.0, 10.0
print(f"Exato:      {juro_real_exato(selic, ipca):.3f}%")       # 4.545%
print(f"Aproximado: {juro_real_aproximado(selic, ipca):.3f}%")  # 5.000%
```

```
Exato:      4.545%
Aproximado: 5.000%
```

A diferença (≈0,45 ponto percentual) parece pequena, mas em taxas altas ela cresce — é por isso que a fórmula aproximada (`selic - ipca`) só deve ser usada como estimativa rápida de cabeça, nunca em cálculo financeiro de precisão. O operador de potência (`**`) não aparece aqui, mas a estrutura da fórmula — divisão encadeada com parênteses — é o mesmo tipo de expressão aritmética do Capítulo 5.1, só que aplicada com rigor.

Nível	Operadores (do que "amarra" mais para o que amarra menos)	Exemplo
1	`**` (potência)	`2 ** 3 ** 2` = 512
2	`+x`, `-x` (sinal), `not`	`-2 ** 2` = −4
3	`*`, `/`, `//`, `%`	`10 + 6 / 2` = 13,0
4	`+`, `-`	

Nível	Operadores (do que "amarra" mais para o que amarra menos)	Exemplo
5	`<`, `<=`, `>`, `>=`, `==`, `!=`, `is`, `in`	`1 < 2 == 2` encadeia
6	`and`	
7	`or`	`a or b and c` = `a or (b and c)`

Aprofundando a teoria — divisão inteira e módulo com números negativos; `and`/`or` devolvem operandos
`//` arredonda em direção a **menos infinito** (não em direção a zero) e `%` herda o sinal do divisor, de modo que vale sempre `a == (a // b) * b + a % b`. E `and`/`or` não devolvem sempre `True`/`False`: devolvem **um dos operandos**, o que permite a expressão "valor padrão": `x or padrao`.

```
print("-7 // 2 =", -7 // 2, "| -7 % 2 =", -7 % 2, "| divmod(-7, 2) =", divmod(-7, 2), "|
identidade:", (-7 // 2) * 2 + (-7 % 2))
print("int(-7 / 2) =", int(-7 / 2), "(trunca para zero) -> diferente de -7 // 2")

taxa_informada = ""                                  # vazio => usar o padrão
print("'' or 'sem dado':", repr(taxa_informada or "sem dado"), "| 'Brasil' and 'BR':",
repr("Brasil" and "BR"), "| 0 or None:", repr(0 or None))
print("2 ** 3 ** 2 =", 2 ** 3 ** 2, "| (2 ** 3) ** 2 =", (2 ** 3) ** 2, "| -2 ** 2 =", -2
** 2)
```

```
-7 // 2 = -4 | -7 % 2 = 1 | divmod(-7, 2) = (-4, 1) | identidade: -7
int(-7 / 2) = -3 (trunca para zero) -> diferente de -7 // 2
'' or 'sem dado': 'sem dado' | 'Brasil' and 'BR': 'BR' | 0 or None: None
2 ** 3 ** 2 = 512 | (2 ** 3) ** 2 = 64 | -2 ** 2 = -4
```

Atenção — erro comum
O `or` como "valor padrão" tem um efeito colateral: `taxa or 5.0` trata `0.0` como "ausente", porque zero é falso. Se zero é um valor *legítimo* (uma taxa de 0%), use `if taxa is None: taxa = 5.0`.

Capítulo 6 — Strings em detalhe

6.1. Strings são sequências imutáveis

Uma string é uma **sequência ordenada de caracteres** — e, como toda sequência em Python, pode ser indexada e fatiada com a mesma sintaxe que veremos em listas no próximo capítulo:

```
pais = "Brasil"
print(pais[0])     # "B" (primeiro caractere, índice 0)
print(pais[-1])    # "l" (último caractere)
print(pais[0:3])   # "Bra" (fatiamento)
```

```
B
l
Bra
```

O ponto crucial: strings são **imutáveis**. Uma vez criada, você não pode alterar um caractere dentro dela:

```
pais[0] = "b"  # TypeError! strings não suportam atribuição de item
```

Qualquer operação que "parece" modificar uma string na verdade **cria uma string nova**:

```
pais_minusculo = pais.lower()  # cria uma string nova; "pais" continua "Brasil"
```

6.2. Concatenação e f-strings

```
# Concatenação com +
pais = "Brasil"
ano = 2026
frase = pais + " em " + str(ano)  # "Brasil em 2026"
# str() converte número para texto — obrigatório, já que + entre str e int dá erro

# F-string (a forma recomendada e mais legível)
frase2 = f"{pais} em {ano}"
print(frase2)  # "Brasil em 2026"
```

```
Brasil em 2026
```

F-strings (introduzidas no Python 3.6) permitem embutir expressões inteiras dentro de `{}`, não só variáveis:

```
selic = 10.5
ipca = 4.5
print(f"Juro real aproximado: {selic - ipca:.1f}%")  # calcula e formata na mesma linha
```

```
Juro real aproximado: 6.0%
```

6.3. Mini-linguagem de formatação

O formato dentro de `{valor:especificação}` segue uma mini-linguagem própria de Python:

Especificação	Efeito	Exemplo
`:.2f`	2 casas decimais	`f"{0.1667:.2f}"` → `"0.17"`
`:.0f`	Sem casas decimais	`f"{0.1667:.0f}"` → `"0"`
`:,`	Separador de milhar	`f"{1000000:,}"` → `"1,000,000"`
`:>10`	Alinha à direita em 10 espaços	útil para alinhar colunas de números
`:%`	Formata como percentual	`f"{0.045:.1%}"` → `"4.5%"`

```
ipca = 0.1667
print(f"IPCA: {ipca:.2f}%")     # "IPCA: 0.17%"
print(f"IPCA: {ipca:.1f}%")     # "IPCA: 0.2%"

taxa = 0.045
print(f"Taxa: {taxa:.1%}")      # "Taxa: 4.5%" — multiplica por 100 e adiciona %
```

```
IPCA: 0.17%
IPCA: 0.2%
Taxa: 4.5%
```

6.4. Métodos úteis de string

```
texto = "  Brasil  "
texto.upper()        # "  BRASIL  "
texto.lower()        # "  brasil  "
texto.strip()        # "Brasil" (remove espaços das pontas — essencial ao limpar dados
```

```
importados)
texto.replace("a", "o")  # "  Brosil  "
texto.split("a")      # ["  Br", "sil  "]
",".join(["a", "b", "c"])  # "a,b,c" — o inverso de split
```

`strip()` e `split()` merecem destaque especial: ao importar dados econômicos de um CSV ou de uma API, é comum vir texto com espaços indesejados (`" Brasil "`) ou valores concatenados (`"2024-Q1"`) que precisam ser separados. Esses dois métodos resolvem 90% desses casos.

6.6. Exemplo Resolvido — Relatório formatado com alinhamento

Enunciado: dado o dicionário `pib` abaixo, imprima uma tabela de texto simples com o nome do país alinhado à esquerda em 12 caracteres e o valor alinhado à direita em 8 caracteres, com 1 casa decimal e símbolo de percentual.

```
pib = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1}
```

Solução comentada:

```
pib = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1}

for pais, valor in pib.items():
    print(f"{pais:<12}{valor:>8.1f}%")
```

Saída:

```
Brasil           2.2%
Argentina       -1.8%
Chile            3.1%
```

`{pais:<12}` alinha a string à **esquerda** (`<`) ocupando 12 caracteres — preenchendo com espaços à direita. `{valor:>8.1f}` alinha o número à **direita** (`>`) em 8 caracteres, com 1 casa decimal (`.1f`). Essa combinação — alinhamento + casas decimais — é exatamente o que gera tabelas de texto legíveis em relatórios impressos no terminal, sem precisar de nenhuma biblioteca externa.

> **Aprofundando a teoria — texto, bytes e acentos: Unicode e normalização**
> Uma `str` é uma sequência de **pontos de código Unicode** para gravar em arquivo ou transmitir, ela é convertida em **bytes** por uma codificação (UTF-8 é o padrão atual). Ler um arquivo com a codificação *errada* gera o clássico "mojibake" (`SÃ£o Paulo`). Além disso, um mesmo caractere acentuado pode ter duas representações Unicode (composta e decomposta), e `==` as considera *diferentes* — um problema real ao cruzar nomes vindos de sistemas diferentes.

```
import unicodedata
nome = "São Paulo"
bytes_utf8 = nome.encode("utf-8")
print("UTF-8:", bytes_utf8, "| lido como latin-1 (errado):", bytes_utf8.decode("latin-1"))

composta = "ç"                                  # um único ponto de código
decomposta = unicodedata.normalize("NFD", "ç")  # 'c' + acento separado
print(f"len composta = {len(composta)}, len decomposta = {len(decomposta)} | iguais com ==?
{composta == decomposta}")
print("Após normalizar (NFC):", unicodedata.normalize("NFC", composta) ==
unicodedata.normalize("NFC", decomposta))
print("casefold:", "STRASSE".casefold() == "straße".casefold(), "| lower:",
"STRASSE".lower() == "straße".lower())
```

```
UTF-8: b'S\xc3\xa3o Paulo' | lido como latin-1 (errado): SÃ£o Paulo
len composta = 1, len decomposta = 2 | iguais com ==? False
Após normalizar (NFC): True
casefold: True | lower: False
```

Exemplo resolvido — extraindo campos com expressões regulares (módulo `re`)
Quando o formato de uma string é regular, mas variável, `split` e fatiamento ficam frágeis. Uma **expressão regular** descreve o padrão. Exemplo: datas no formato brasileiro `dd/mm/aaaa`, com validação de formato:

```
import re
padrao = re.compile(r"(\d{2})/(\d{2})/(\d{4})")
for texto in ["31/12/2025", "5/1/2026", "01-02-2026"]:
    m = padrao.fullmatch(texto)
    print(f"{texto!r:14} -> {('dia %s, mês %s, ano %s' % m.groups()) if m else 'formato
inválido'}")
```

```
'31/12/2025'   -> dia 31, mês 12, ano 2025
'5/1/2026'     -> formato inválido
'01-02-2026'   -> formato inválido
```

Capítulo 7 — Listas

7.1. O que é uma lista

Uma lista é uma coleção **ordenada** e **mutável** de valores.

```
inflacoes = [0.50, 0.70, 0.88, 0.67, 0.58, 0.16]
paises = ["Brasil", "Argentina", "Chile", "Colômbia"]
mista = [1, "texto", True, 3.14]  # tipos diferentes são permitidos, mas evite — dificulta
manutenção
```

7.2. Indexação e fatiamento

```
# Acessando elementos (o índice começa em 0!)
inflacoes[0]    # 0.50  (primeiro)
inflacoes[1]    # 0.70  (segundo)
inflacoes[-1]   # 0.16  (último)
inflacoes[-2]   # 0.58  (penúltimo)

# Fatiamento (slicing) — [inicio:fim:passo]
inflacoes[0:3]     # [0.50, 0.70, 0.88] (índices 0, 1, 2 — o "fim" nunca é incluído)
inflacoes[:3]      # [0.50, 0.70, 0.88] (mesma coisa; início vazio = "desde o começo")
inflacoes[3:]      # [0.67, 0.58, 0.16] (do índice 3 até o fim)
inflacoes[::2]     # [0.50, 0.88, 0.58] (pulando de 2 em 2)
inflacoes[::-1]    # [0.16, 0.58, 0.67, 0.88, 0.70, 0.50] (inverte a ordem)
```

A regra de que o índice final **nunca é incluído** é uma fonte comum de erro em iniciantes (chamado *off-by-one error*). Uma forma de memorizar: `lista[a:b]` tem exatamente `b - a` elementos.

7.3. Mutabilidade e o problema da aliasing

Diferente das strings, listas **são mutáveis**: você pode alterar um elemento no lugar, sem criar uma lista nova. Isso, combinado com o modelo de "variável como rótulo" do Capítulo 3, gera um comportamento que surpreende muitos iniciantes:

```
inflacoes_2024 = [0.5, 0.7, 0.9]
inflacoes_copia = inflacoes_2024   # isso NÃO cria uma cópia! copia só o rótulo

inflacoes_copia.append(1.2)
print(inflacoes_2024)  # [0.5, 0.7, 0.9, 1.2] — mudou também!
```

```
[0.5, 0.7, 0.9, 1.2]
```

`inflacoes_copia = inflacoes_2024` não copia os dados: cria um segundo rótulo apontando para o **mesmo** objeto lista na memória. Alterar um afeta o outro, porque só existe uma lista. Isso é chamado de **aliasing**. Para criar uma cópia de verdade:

```
inflacoes_copia = inflacoes_2024.copy()   # ou list(inflacoes_2024), ou inflacoes_2024[:]
inflacoes_copia.append(1.2)
print(inflacoes_2024)   # [0.5, 0.7, 0.9] — agora não muda
print(inflacoes_copia)  # [0.5, 0.7, 0.9, 1.2]
```

```
[0.5, 0.7, 0.9, 1.2]
[0.5, 0.7, 0.9, 1.2, 1.2]
```

(Isso é uma cópia **rasa** — "shallow copy". Se a lista contiver outras listas dentro, os elementos internos ainda seriam compartilhados. Para esse caso existe `copy.deepcopy()`, mas isso é raro no dia a dia de análise de dados.)

7.4. Métodos de lista

```
numeros = [1, 2, 3]
numeros.append(4)        # [1, 2, 3, 4]  (adiciona no fim)
numeros.insert(0, 0)     # [0, 1, 2, 3, 4] (insere na posição 0)
numeros.remove(2)        # [0, 1, 3, 4] (remove a primeira ocorrência do VALOR 2)
numeros.pop()            # remove e retorna o último elemento
numeros.pop(0)           # remove e retorna o elemento do índice 0
len(numeros)             # tamanho da lista
sum(inflacoes)           # soma
max(inflacoes)           # maior valor
min(inflacoes)           # menor valor
sorted(inflacoes)        # nova lista ordenada (crescente) — não altera a original
sorted(inflacoes, reverse=True)  # decrescente
```

Repare a diferença entre `remove()` (remove por **valor**) e `pop()` (remove por **índice**, e ainda devolve o valor removido) — os nomes parecidos costumam confundir.

7.6. Exemplo Resolvido — Corrigindo outliers sem afetar a lista original

Enunciado: dada uma lista de inflações mensais, escreva uma função `capar_outliers` que devolva uma **lista nova** onde qualquer valor acima de um limite seja substituído pelo próprio limite — sem alterar a lista original (evitando o problema de aliasing visto na Seção 7.3).

```
inflacoes = [0.50, 0.70, 3.50, 0.67, 0.58, 0.16]  # 3.50 parece um erro de digitação
```

Solução comentada:

```
def capar_outliers(dados, limite):
    return [valor if valor <= limite else limite for valor in dados]

inflacoes = [0.50, 0.70, 3.50, 0.67, 0.58, 0.16]
inflacoes_corrigidas = capar_outliers(inflacoes, limite=1.0)
```

```
print(inflacoes)              # [0.5, 0.7, 3.5, 0.67, 0.58, 0.16] — inalterada
print(inflacoes_corrigidas)   # [0.5, 0.7, 1.0, 0.67, 0.58, 0.16]
```

```
[0.5, 0.7, 3.5, 0.67, 0.58, 0.16]
[0.5, 0.7, 1.0, 0.67, 0.58, 0.16]
```

O ponto central: a list comprehension **sempre constrói uma lista nova**, então não há risco de aliasing aqui — diferente de `inflacoes.append(...)` ou `inflacoes[2] = ...`, que alterariam o objeto original no lugar. Sempre que a intenção for "criar uma versão modificada, preservando os dados originais", prefira uma expressão que gere uma lista nova a um método que modifique a lista existente.

Aprofundando a teoria — o custo das operações em listas e a escolha entre lista e tupla
A lista é um **array dinâmico**: acesso por índice e `append` são rápidos (tempo constante); inserir ou remover no **início** exige deslocar todos os elementos (tempo proporcional ao tamanho); procurar um valor (`x in lista`) percorre a lista (tempo proporcional). Saber isso explica por que `x in lista` num laço sobre milhares de elementos fica lento — e por que um conjunto (`set`) ou dicionário resolve melhor (capítulo 8). A **tupla** é uma lista **imutável**: serve para registros que não devem mudar (`(ano, trimestre)`) e pode ser usada como chave de dicionário.

```
import copy
meses = ["jan", "fev", "mar", "abr", "mai", "jun", "jul", "ago"]
print("meses[::2] =", meses[::2], "| meses[::-1] =", meses[::-1][:3], "... | meses[-3:] =",
meses[-3:])

base = [["Brasil", 2.2], ["Chile", 3.1]]
rasa = list(base)                  # cópia rasa: copia a lista externa, compartilha as
listas internas
funda = copy.deepcopy(base)        # cópia profunda: copia tudo
base[0][1] = 9.9
print("Após alterar base[0][1]:  rasa =", rasa, "| funda =", funda)

paises = [("Brasil", 2.2), ("Chile", 3.1), ("Peru", 2.5), ("México", 1.9)]
print("Ordenado por PIB (decrescente):", sorted(paises, key=lambda par: par[1],
reverse=True))
print("A lista original não mudou:", paises[0], "(sorted devolve nova lista; list.sort()
altera a original)")
```

```
meses[::2] = ['jan', 'mar', 'mai', 'jul'] | meses[::-1] = ['ago', 'jul', 'jun'] ... |
meses[-3:] = ['jun', 'jul', 'ago']
Após alterar base[0][1]:  rasa = [['Brasil', 9.9], ['Chile', 3.1]] | funda = [['Brasil',
2.2], ['Chile', 3.1]]
Ordenado por PIB (decrescente): [('Chile', 3.1), ('Peru', 2.5), ('Brasil', 2.2), ('México',
1.9)]
A lista original não mudou: ('Brasil', 2.2) (sorted devolve nova lista; list.sort() altera
a original)
```

Atenção — erro comum
`sort()` devolve `None` (altera no lugar); escrever `lista = lista.sort()` apaga a lista. E a ordenação do Python é **estável**: elementos com a mesma chave mantêm a ordem relativa original — por isso se pode ordenar por um critério secundário primeiro e pelo principal depois.

Capítulo 8 — Dicionários

8.1. O que é um dicionário e por que ele é rápido

Um dicionário armazena pares **chave → valor**. Diferente de uma lista, onde você acessa por posição numérica, num dicionário você acessa por uma chave que você mesmo escolhe (um nome, uma sigla, uma data).

```
pib = {
    "Brasil": 2.2,
    "Argentina": -1.8,
    "Chile": 3.1,
    "Colômbia": 2.8
}
```

Internamente, um dicionário Python é implementado como uma **tabela hash**: cada chave passa por uma função matemática (a *hash function*) que calcula diretamente onde o valor está guardado na memória. Isso significa que buscar `pib["Brasil"]` é praticamente instantâneo, independentemente de o dicionário ter 10 ou 10 milhões de entradas — bem diferente de procurar um valor numa lista, que no pior caso exige percorrer item por item.

Como isso funciona, em mais detalhe. A função hash (`hash("Brasil")` em Python) transforma a chave num número inteiro grande; esse número, reduzido ao tamanho da tabela interna (via resto da divisão), aponta diretamente para o "balde" (*bucket*) onde o par chave-valor deveria estar. Isso é o que dá complexidade $O(1)$ — tempo constante, não crescente com o tamanho dos dados — para buscar, inserir ou remover, em contraste com a lista, cuja busca por valor é $O(n)$ (no pior caso, percorre todos os n elementos). Duas consequências práticas dessa implementação:

- **Colisões existem e são tratadas automaticamente.** Duas chaves diferentes podem, por acaso, mapear para o mesmo balde (`hash("Brasil") % tamanho_tabela == hash("Chile") % tamanho_tabela`); o Python resolve isso internamente, mas é o motivo de a complexidade $O(1)$ ser uma *média*, não uma garantia absoluta.
- **A chave precisa ser hasheável, e portanto imutável.** É por isso que listas não podem ser chave de dicionário (`TypeError: unhashable type: 'list'`) — se o conteúdo da chave pudesse mudar depois de guardada, seu hash mudaria, e o Python não conseguiria mais encontrá-la no balde certo. Tuplas (Capítulo 11.6), strings e números podem ser chave; listas e dicionários, não.

Na prática: **use dicionário sempre que a busca for por um identificador nomeado** (país, código, data), e lista quando a ordem sequencial for o que importa.

8.2. Acessando e modificando

```
print(pib["Brasil"])       # 2.2
print(pib.get("Brasil"))   # 2.2 — forma segura, não quebra se a chave não existir
print(pib.get("Uruguai", 0.0))  # 0.0 — valor padrão quando a chave não existe

# pib["Uruguai"] sem o .get() geraria um KeyError

# Adicionar/modificar
pib["Peru"] = 3.5     # adiciona uma chave nova
pib["Brasil"] = 2.5   # atualiza uma chave existente
```

```
2.2
2.2
0.0
```

8.3. Métodos e iteração

```
pib.keys()      # todas as chaves
pib.values()    # todos os valores
pib.items()     # pares (chave, valor)

for pais, valor in pib.items():
    print(f"{pais}: {valor}%")
```

```
Brasil: 2.5%
Argentina: -1.8%
Chile: 3.1%
Colômbia: 2.8%
Peru: 3.5%
```

8.4. Dicionários aninhados

É comum, em dados econômicos, ter mais de uma dimensão — por exemplo, PIB por país **e** por ano. A solução natural é um dicionário de dicionários:

```
pib_por_ano = {
    "Brasil": {"2023": 2.9, "2024": 3.1, "2025": 2.2},
    "Chile":  {"2023": 0.2, "2024": 2.6, "2025": 3.1},
}

print(pib_por_ano["Brasil"]["2024"])  # 3.1

for pais, series in pib_por_ano.items():
    for ano, valor in series.items():
        print(f"{pais} em {ano}: {valor}%")
```

```
3.1
Brasil em 2023: 2.9%
Brasil em 2024: 3.1%
Brasil em 2025: 2.2%
Chile em 2023: 0.2%
Chile em 2024: 2.6%
Chile em 2025: 3.1%
```

Vale adiantar: na Apostila 2 (pandas), esse tipo de estrutura aninhada normalmente vira um `DataFrame`, que é mais adequado para manipular tabelas de verdade — mas entender o dicionário aninhado ajuda a entender o que o pandas faz por baixo dos panos.

8.5. Lista vs. dicionário — quando usar qual

Situação	Estrutura recomendada
Sequência de valores na ordem em que ocorreram (ex: IPCA mês a mês)	Lista
Valores identificados por nome (ex: PIB por país)	Dicionário
Preciso acessar por posição (“o terceiro valor”)	Lista
Preciso acessar por identificador (“o valor do Chile”)	Dicionário

8.6. Exemplo Resolvido — Combinando dois dicionários

Enunciado: dados dois dicionários com o PIB do mesmo conjunto de países em anos diferentes, escreva uma função `variacao_percentual` que retorne um **novo dicionário** `{país: variação}` com a diferença entre os dois anos, país por país.

```
pib_2024 = {"Brasil": 3.1, "Chile": 2.6, "Peru": 3.3}
pib_2025 = {"Brasil": 2.2, "Chile": 3.1, "Peru": 2.9}
```

Solução comentada:

```
def variacao_percentual(dados_ano1, dados_ano2):
    resultado = {}
    for pais in dados_ano1:
        resultado[pais] = dados_ano2[pais] - dados_ano1[pais]
    return resultado

pib_2024 = {"Brasil": 3.1, "Chile": 2.6, "Peru": 3.3}
pib_2025 = {"Brasil": 2.2, "Chile": 3.1, "Peru": 2.9}

print(variacao_percentual(pib_2024, pib_2025))
# {'Brasil': -0.9, 'Chile': 0.5, 'Peru': -0.4}
```

```
{'Brasil': -0.8999999999999999, 'Chile': 0.5, 'Peru': -0.3999999999999999}
```

Repare que `for pais in dados_ano1` itera diretamente sobre as **chaves** do dicionário (isso é equivalente a `for pais in dados_ano1.keys()`, mas mais direto). A função pressupõe que os dois dicionários têm exatamente as mesmas chaves; se isso não for garantido, o acesso `dados_ano2[pais]` poderia lançar um `KeyError` — o Capítulo 13 mostra como blindar esse tipo de acesso.

> **Aprofundando a teoria — por que o dicionário é rápido: tabela hash e chaves imutáveis**
> Um dicionário calcula, para cada chave, um **hash** (um número inteiro) que indica em que "gaveta" guardar o valor. Buscar uma chave é calcular o hash e ir direto à gaveta: o tempo **não cresce** com o tamanho do dicionário (em média). Isso exige que a chave seja **imutável** (strings, números, tuplas): se uma chave mudasse, seu hash mudaria e o valor "se perderia". Por isso `{[1, 2]: "x"}` dá `TypeError: unhashable type: 'list'`. Conjuntos (`set`) usam a mesma ideia e servem para testar pertencimento e eliminar duplicados.

```
from collections import Counter, defaultdict

classificacoes = ["AAA", "BBB", "BB", "AAA", "A", "BBB", "AAA", "B"]
contagem = Counter(classificacoes)
print("Counter:", contagem, "| mais comum:", contagem.most_common(1))

por_setor = defaultdict(list)                    # valor padrão: lista vazia (não precisa
checar se a chave existe)
for empresa, setor in [("Alfa", "Energia"), ("Beta", "Varejo"), ("Gama", "Energia")]:
    por_setor[setor].append(empresa)
print("defaultdict:", dict(por_setor))

pib = {"Brasil": 2.2, "Chile": 3.1}
extra = {"Peru": 2.5, "Chile": 3.3}
print("Mesclando com '|' (o da direita vence):", pib | extra)
print("get com padrão:", pib.get("Argentina", "sem dado"), "| 'Chile' in pib:", "Chile" in
pib)
try:
    {[1, 2]: "x"}
```

```
except TypeError as erro:
    print("Chave mutável:", erro)
```

```
Counter: Counter({'AAA': 3, 'BBB': 2, 'BB': 1, 'A': 1, 'B': 1}) | mais comum: [('AAA', 3)]
defaultdict: {'Energia': ['Alfa', 'Gama'], 'Varejo': ['Beta']}
Mesclando com '|' (o da direita vence): {'Brasil': 2.2, 'Chile': 3.3, 'Peru': 2.5}
get com padrão: sem dado | 'Chile' in pib: True
Chave mutável: unhashable type: 'list'
```

Capítulo 9 — Condicionais

9.1. if / elif / else

```
ipca = 0.88

if ipca > 1.0:
    print("Inflação alta")
elif ipca > 0.5:
    print("Inflação moderada")
elif ipca > 0.2:
    print("Inflação controlada")
else:
    print("Deflação")
```

```
Inflação moderada
```

O Python avalia as condições **em ordem**, de cima para baixo, e executa apenas o **primeiro** bloco cuja condição seja verdadeira — mesmo que uma condição posterior também fosse verdadeira. Por isso a ordem de um `elif` importa.

9.2. Truthiness — o que conta como "verdadeiro"

Em condicionais, Python aceita qualquer valor, não apenas `True`/`False` — e converte implicitamente para booleano. Essa conversão segue uma regra: valores "vazios" ou "zero" são considerados falsos (*falsy*); praticamente tudo o mais é considerado verdadeiro (*truthy*).

```
lista_vazia = []
lista_com_dados = [0.5]

if lista_vazia:
    print("tem dados")
else:
    print("lista vazia")   # imprime isso — lista vazia é falsy

if 0:
    print("nunca executa")   # 0 é falsy
if 0.0:
    print("nunca executa")   # 0.0 é falsy
if "":
    print("nunca executa")   # string vazia é falsy
if "0":
    print("executa!")        # string "0" (não vazia) é truthy!
```

```
lista vazia
executa!
```

O último exemplo é uma armadilha clássica: a string `"0"` é truthy, mesmo representando "zero", porque o que importa é ela **não estar vazia**.

9.3. Operador ternário

```
status = "Alta" if ipca > 0.5 else "Baixa"
```

Isso é equivalente a um `if/else` de quatro linhas, mas condensado numa expressão — útil quando o valor precisa ser atribuído diretamente a uma variável.

9.4. Combinando condições

```
pib = -0.5
desemprego = 12.0

if pib < 0 and desemprego > 10:
    print("País em recessão com alto desemprego")
elif pib < 0 or desemprego > 10:
    print("Um dos indicadores está negativo")
```

```
País em recessão com alto desemprego
```

9.5. Exemplo Resolvido — Tratando dado ausente antes de comparar

Enunciado: é comum que uma série econômica tenha meses sem dado disponível, representados como `None` (o "vazio" do Python). Escreva uma função `classifica_com_dado_ausente(ipca)` que retorne `"sem dado"` se `ipca` for `None`, e classifique normalmente (`"alta"` se > 0.5, senão `"controlada"`) caso contrário — **sem** deixar o programa quebrar ao comparar `None` com um número.

```
valores = [0.6, None, 0.3, 0.9]
```

Solução comentada:

```
def classifica_com_dado_ausente(ipca):
    if ipca is None:          # sempre confira None ANTES de comparar com números
        return "sem dado"
    elif ipca > 0.5:
        return "alta"
    else:
        return "controlada"

valores = [0.6, None, 0.3, 0.9]
for v in valores:
    print(classifica_com_dado_ausente(v))
# alta / sem dado / controlada / alta
```

```
alta
sem dado
controlada
alta
```

O detalhe crítico é a **ordem**: `ipca is None` precisa vir antes de `ipca > 0.5`, porque `None > 0.5` gera um `TypeError` em Python (não existe comparação de ordem entre `None` e um número — diferente de outras linguagens onde isso silenciosamente viraria `False`). Usar `is None` (em vez de `== None`) é a forma idiomática recomendada, porque `None` é um valor único e especial na linguagem, e `is` verifica identidade exata, não apenas igualdade.

Aprofundando a teoria — verdadeiro e falso em Python (truthiness) e o `match`
Em `if x:`, o Python converte `x` em booleano. São **falsos**: `False`, `None`, `0`, `0.0`, `""` (string vazia), `[]`, `{}`, `()`, `set()`. Todo o resto é verdadeiro — inclusive `"0"` e `[0]`. Para decisões em vários casos mutuamente exclusivos, o Python 3.10 trouxe o `match`/`case`, mais legível que uma cadeia de `elif` quando se comparam *padrões* de um mesmo valor.

```
for valor in [0, 0.0, "", "0", [], [0], None, {}, {"a": 1}]:
    print(f"{valor!r:>8} -> {'verdadeiro' if valor else 'falso'}")

def faixa_de_renda(salarios_minimos):
    if salarios_minimos is None:                        # só None é "dado ausente"; 0 é um
valor legítimo
        return "sem informação"
    if salarios_minimos < 2:
        return "baixa"
    if salarios_minimos < 5:
        return "média"
    return "alta"

def descreve_tipo(valor):
    match valor:
        case int() | float():
            return "número"
        case str():
            return "texto"
        case [primeiro, *resto]:
            return f"lista começando em {primeiro!r} (+{len(resto)} itens)"
        case {"pais": nome}:
            return f"registro do país {nome}"
        case _:
            return "outro"

print([faixa_de_renda(x) for x in (None, 0, 3, 12)])
print([descreve_tipo(v) for v in (3.5, "texto", [1, 2, 3], {"pais": "Brasil"}, None)])
```

```
       0 -> falso
     0.0 -> falso
      '' -> falso
     '0' -> verdadeiro
      [] -> falso
     [0] -> verdadeiro
    None -> falso
      {} -> falso
{'a': 1} -> verdadeiro
['sem informação', 'baixa', 'média', 'alta']
['número', 'texto', 'lista começando em 1 (+2 itens)', 'registro do país Brasil', 'outro']
```

Capítulo 10 — Laços de repetição

10.1. `for` — percorrendo sequências (iteráveis)

Listas, strings, dicionários e vários outros tipos em Python compartilham a propriedade de serem **iteráveis**: dá para percorrê-los item por item com `for`. Isso não é coincidência — é um protocolo geral da linguagem, e por isso a sintaxe do `for` é a mesma independentemente do tipo de dado percorrido:

```python
# Percorrer lista
inflacoes = [0.50, 0.70, 0.88, 0.67]
for valor in inflacoes:
    print(f"IPCA: {valor}%")

# Percorrer com índice, usando enumerate()
for i, valor in enumerate(inflacoes):
    print(f"Mês {i+1}: {valor}%")

# Percorrer dicionário
pib = {"Brasil": 2.2, "Chile": 3.1}
for pais, valor in pib.items():
    print(f"{pais}: {valor}%")

# range() gera uma sequência de números — útil para repetir N vezes
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4 (começa em 0, para antes de 5)

for i in range(1, 13):
    print(f"Mês {i}")  # 1 a 12
```

```
IPCA: 0.5%
IPCA: 0.7%
IPCA: 0.88%
IPCA: 0.67%
Mês 1: 0.5%
Mês 2: 0.7%
Mês 3: 0.88%
Mês 4: 0.67%
Brasil: 2.2%
Chile: 3.1%
0
1
2
3
4
Mês 1
Mês 2
Mês 3
Mês 4
Mês 5
Mês 6
Mês 7
Mês 8
Mês 9
Mês 10
Mês 11
Mês 12
```

10.2. `while` — repetir enquanto uma condição for verdadeira

```python
saldo = 1000
taxa = 0.01
anos = 0

while saldo < 2000:
    saldo *= (1 + taxa)
    anos += 1

print(f"Leva {anos} anos para dobrar")
```

```
# CUIDADO: loop infinito! Sempre garanta que a condição
# vai se tornar False em algum momento — aqui, saldo cresce a cada
# iteração e eventualmente ultrapassa 2000.
```

```
Leva 70 anos para dobrar
```

Regra prática: use `for` quando souber, de antemão, sobre o que está iterando (uma lista, um intervalo fixo). Use `while` quando a repetição depende de uma condição que só se resolve durante a execução (como "até o saldo dobrar" — não se sabe quantas iterações de antemão).

10.3. `break` e `continue`

```
# break — interrompe o loop imediatamente
for valor in inflacoes:
    if valor > 0.8:
        print("Alta detectada, parando")
        break
    print(valor)

# continue — pula para a próxima iteração, sem executar o resto do bloco
for valor in inflacoes:
    if valor < 0.3:
        continue  # pula valores baixos
    print(valor)
```

```
0.5
0.7
Alta detectada, parando
0.5
0.7
0.88
0.67
```

10.4. Nota sobre desempenho: laços vs. vetorização

Um `for` em Python percorre elementos um a um, o que é intuitivo mas relativamente lento para volumes grandes de dados (milhões de linhas). Bibliotecas como `numpy` e `pandas` (Apostila 2) permitem **vetorizar** a mesma operação — aplicá-la a um array inteiro de uma vez, usando código otimizado em C por baixo dos panos. Por enquanto, `for` é a ferramenta certa para aprender os conceitos; na prática profissional com bases de dados grandes, você vai preferir a versão vetorizada sempre que ela existir.

10.5. Exemplo Resolvido — Trajetória de patrimônio mês a mês

Enunciado: um investidor aplica R$ 500 por mês (aporte constante) a uma taxa de 0,8% ao mês. Construa, com um `for`, a lista com o saldo acumulado ao final de cada um dos 12 meses (aporte entra **antes** de render no mês).

Solução comentada:

```
def trajetoria_patrimonio(aporte_mensal, taxa, meses):
    saldo = 0.0
    trajetoria = []
    for _ in range(meses):
        saldo += aporte_mensal        # aporte do mês entra primeiro
        saldo *= (1 + taxa)           # depois rende sobre o saldo já com o aporte
        trajetoria.append(round(saldo, 2))
    return trajetoria

historico = trajetoria_patrimonio(aporte_mensal=500, taxa=0.008, meses=12)
```

```
for mes, saldo in enumerate(historico, start=1):
    print(f"Mês {mes:2d}: R$ {saldo:.2f}")
```

```
Mês  1: R$ 504.00
Mês  2: R$ 1012.03
Mês  3: R$ 1524.13
Mês  4: R$ 2040.32
Mês  5: R$ 2560.64
Mês  6: R$ 3085.13
Mês  7: R$ 3613.81
Mês  8: R$ 4146.72
Mês  9: R$ 4683.89
Mês 10: R$ 5225.37
Mês 11: R$ 5771.17
Mês 12: R$ 6321.34
```

O uso de _ como nome de variável no `for _ in range(meses)` é uma convenção do Python para dizer "este valor existe, mas eu não preciso dele" — aqui, só precisamos repetir o bloco `meses` vezes, sem usar o índice diretamente. Note também `enumerate(historico, start=1)`: o parâmetro `start` desloca a contagem para começar em 1 em vez de 0, evitando o `+1` manual usado em exemplos anteriores.

Aprofundando a teoria — iteradores e geradores: o que `for` faz de verdade
Um laço `for x in colecao` pede à coleção um **iterador** e chama repetidamente `next()` até uma exceção `StopIteration`. Qualquer objeto que ofereça essa interface é **iterável**: listas, strings, dicionários, arquivos, `range`. Um **gerador** (função com `yield`) produz os valores *sob demanda*, um de cada vez, sem guardar todos na memória — essencial para arquivos enormes. Funções como `enumerate`, `zip`, `reversed` e `range` são iteradores "preguiçosos" (*lazy*).

```
def metodo_de_newton_sqrt(x, tolerancia=1e-12):
    """Raiz quadrada por Newton: x_{n+1} = (x_n + x/x_n) / 2, e o `while` roda até
convergir."""
    estimativa, passos = x, 0
    while abs(estimativa ** 2 - x) > tolerancia:
        estimativa = (estimativa + x / estimativa) / 2
        passos += 1
    return estimativa, passos

raiz, passos = metodo_de_newton_sqrt(2)
print(f"sqrt(2) ≈ {raiz:.12f} em {passos} iterações (math.sqrt: {2 ** 0.5:.12f})")

def meses_do_ano():                    # gerador: entrega um valor por vez, sob demanda
    for numero, nome in enumerate(["jan", "fev", "mar"], start=1):
        yield numero, nome

gerador = meses_do_ano()
print(next(gerador), next(gerador), "| restante:", list(gerador))

anos = [2023, 2024, 2025]
pibs = [2.9, 3.1, 2.2]
for i, (ano, pib) in enumerate(zip(anos, pibs)):
    print(i, ano, pib)
for n in range(10, 0, -3):
    print("n =", n, end="; ")
print()
for tentativa in range(3):
    if tentativa == 5:
        break
```

```
else:
    print("o 'else' do for roda quando NÃO houve break")
```

```
sqrt(2) ≈ 1.414213562373 em 5 iterações (math.sqrt: 1.414213562373)
(1, 'jan') (2, 'fev') | restante: [(3, 'mar')]
0 2023 2.9
1 2024 3.1
2 2025 2.2
n = 10; n = 7; n = 4; n = 1;
o 'else' do for roda quando NÃO houve break
```

Capítulo 11 — Funções

11.1. Por que funções importam além de "reaproveitar código"

Uma função é um bloco de código reutilizável — mas o motivo mais importante para usá-las, em um contexto de análise econômica, não é apenas evitar repetição: é **isolar e nomear uma etapa do raciocínio**. Uma função bem nomeada, como `taxa_juros_real(selic, ipca)`, documenta a si mesma; um bloco solto de operações não.

Funções também são a base da **reprodutibilidade**: se o cálculo de uma variável está dentro de uma função testável, você pode verificar que ela produz o resultado certo isoladamente, sem rodar todo o script.

```
def nome_da_funcao(parametro1, parametro2):
    """Docstring: explica o que a função faz"""
    resultado = parametro1 + parametro2
    return resultado
```

11.2. Função sem parâmetros e sem retorno

```
def saudacao():
    print("Bem-vindo ao analisador econômico")

saudacao()
```

```
Bem-vindo ao analisador econômico
```

11.3. Parâmetros e retorno

```
def calcular_cambio_real(pib, cambio):
    """Calcula taxa de câmbio real simplificada"""
    return cambio * (1 + pib / 100)

cambio_real = calcular_cambio_real(3.2, 5.45)
print(f"Câmbio real ajustado: {cambio_real:.2f}")
```

```
Câmbio real ajustado: 5.62
```

11.4. Vários parâmetros

```
def taxa_juros_real(selic, ipca_esperado):
    """Calcula taxa de juros real (Fischer simplificado)"""
    return selic - ipca_esperado

juro_real = taxa_juros_real(10.5, 4.5)
print(f"Juro real: {juro_real:.1f}%")
```

```
Juro real: 6.0%
```

11.5. Parâmetros com valor padrão — e uma armadilha a evitar

```python
def pib_per_capita(pib_total, populacao, unidade="mil"):
    """Calcula PIB per capita com unidade configurável"""
    resultado = pib_total / populacao
    if unidade == "mil":
        return resultado / 1000
    elif unidade == "milhao":
        return resultado / 1_000_000
    return resultado

print(pib_per_capita(2_000_000_000_000, 214_000_000))
print(pib_per_capita(2_000_000_000_000, 214_000_000, unidade="milhao"))
```

```
9.345794392523365
0.009345794392523366
```

Valores padrão são convenientes, mas há uma armadilha conhecida: **nunca use uma lista ou dicionário como valor padrão**. Por causa do modelo de memória visto no Capítulo 3, o objeto padrão é criado **uma única vez**, na definição da função — e não a cada chamada:

```python
# ERRADO — bug clássico
def adiciona_pais(pais, lista=[]):
    lista.append(pais)
    return lista

print(adiciona_pais("Brasil"))     # ["Brasil"]
print(adiciona_pais("Chile"))      # ["Brasil", "Chile"] — a lista "padrão" foi
reaproveitada!

# CORRETO
def adiciona_pais_corrigido(pais, lista=None):
    if lista is None:
        lista = []
    lista.append(pais)
    return lista
```

```
['Brasil']
['Brasil', 'Chile']
```

11.6. Retornando múltiplos valores

```python
def estatisticas_basicas(dados):
    """Retorna média, mínimo e máximo de uma lista"""
    media = sum(dados) / len(dados)
    minimo = min(dados)
    maximo = max(dados)
    return media, minimo, maximo  # na verdade, retorna uma tupla (media, minimo, maximo)

inf = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]
media, min_inf, max_inf = estatisticas_basicas(inf)
print(f"Média: {media:.2f}, Mín: {min_inf:.2f}, Máx: {max_inf:.2f}")
```

```
Média: 0.58, Mín: 0.16, Máx: 0.88
```

11.7. Escopo: variáveis locais vs. globais

Uma variável criada **dentro** de uma função só existe dentro dela (escopo local); ela desaparece quando a função termina e não é visível fora:

```
def calcula_dobro(x):
    resultado = x * 2  # "resultado" é local a esta função
    return resultado

calcula_dobro(5)
print(resultado)  # NameError! "resultado" não existe fora da função
```

Isso é desejável: cada função funciona como uma "caixa fechada", e nomes internos não colidem com nomes usados em outras partes do script — o que torna o comportamento de uma função **previsível** independentemente de quem a chama.

11.8. Exemplo Resolvido — Função pura vs. função com efeito colateral

Enunciado: compare as duas funções abaixo, que aparentemente fazem "a mesma coisa" (adicionar um registro de PIB a uma lista de histórico). Explique por que a primeira é mais segura de usar em um script de análise, e qual problema a segunda pode causar.

```
historico = []

def registra_impura(pais, valor):
    historico.append((pais, valor))   # modifica uma lista de FORA da função
    return historico

def registra_pura(historico_atual, pais, valor):
    return historico_atual + [(pais, valor)]  # retorna uma lista NOVA
```

Solução comentada:

`registra_impura` tem um **efeito colateral**: ela modifica `historico`, uma variável definida fora da função, sem receber isso explicitamente como parâmetro. Isso funciona, mas torna o comportamento da função dependente de um estado externo escondido — se você chamar `registra_impura` em partes diferentes do script, o resultado depende de quantas vezes ela já foi chamada antes, o que dificulta testar a função isoladamente.

`registra_pura` é uma **função pura**: seu resultado depende *apenas* dos parâmetros recebidos, e ela não modifica nada fora de si mesma (`historico_atual + [...]` cria uma lista nova, não altera a original — o mesmo princípio de "criar em vez de mutar" do Exemplo Resolvido do Capítulo 7). Isso a torna previsível e fácil de testar: dado o mesmo input, sempre produz o mesmo output.

```
historico = []
historico = registra_pura(historico, "Brasil", 2.2)
historico = registra_pura(historico, "Chile", 3.1)
print(historico)  # [('Brasil', 2.2), ('Chile', 3.1)]
```

```
[('Brasil', 2.2), ('Chile', 3.1)]
```

Na prática profissional, funções puras são preferidas sempre que possível — especialmente em pipelines de dados, onde rastrear "quem alterou o quê" em funções impuras é uma fonte comum de bugs difíceis de reproduzir.

> **Aprofundando a teoria — funções como objetos: argumentos flexíveis, lambda, closures e memoização**
> Em Python, funções são **objetos**: podem ser guardadas em variáveis, passadas como argumento e retornadas. Isso permite padrões poderosos:

- `*args` e `**kwargs` aceitam um número variável de argumentos posicionais e nomeados; parâmetros depois de `*` são **só nomeados**, o que torna chamadas mais legíveis.
- **lambda**: função anônima de uma expressão, útil como `key=` em `sorted`.
- **closure**: uma função que "lembra" variáveis da função que a criou (`criar_deflator(indice)` devolve uma função).
- **memoização**: guardar resultados de chamadas já feitas para não recalculá-las (`functools.lru_cache`).
- **type hints** e **docstrings** documentam intenção e ajudam ferramentas e colegas.

```python
from functools import lru_cache

def resumo(*valores, casas=2):
    """Média e extremos de quantos valores forem passados. `casas` só pode ser dado por
nome."""
    return round(sum(valores) / len(valores), casas), min(valores), max(valores)

def criar_deflator(indice_acumulado: float):
    def deflacionar(valor_nominal: float) -> float:
        return valor_nominal / indice_acumulado          # 'lembra' o índice (closure)
    return deflacionar

para_reais_de_2020 = criar_deflator(1.35)

@lru_cache(maxsize=None)
def fibonacci(n: int) -> int:
    return n if n < 2 else fibonacci(n - 1) + fibonacci(n - 2)

print("resumo(3, 5, 10, casas=1):", resumo(3, 5, 10, casas=1))
print("R$ 1.350 nominais em reais de 2020:", round(para_reais_de_2020(1350), 2))
print("fibonacci(80) =", fibonacci(80), "| acertos de cache:", fibonacci.cache_info().hits)
```

```
resumo(3, 5, 10, casas=1): (6.0, 3, 10)
R$ 1.350 nominais em reais de 2020: 1000.0
fibonacci(80) = 23416728348467685 | acertos de cache: 78
```

Atenção — erro comum
Argumento padrão mutável. `def f(x, lista=[])` cria a lista *uma única vez*, na definição da função; todas as chamadas compartilham a mesma lista (a armadilha da seção 11.5). O padrão correto é `lista=None` e criar a lista dentro da função.

Capítulo 12 — List comprehension

12.1. Sintaxe e motivação

List comprehension é uma forma concisa de criar listas a partir de outra sequência, condensando em uma linha o que um `for` levaria três ou quatro linhas para fazer. É amplamente usada em código Python idiomático, inclusive dentro do próprio pandas.

```python
# Sintaxe: [expressao for item in lista if condicao]

inf = [0.5, 0.7, 0.88, 0.67]

# Aplicar uma transformação a cada elemento
dobro = [x * 2 for x in inf]          # [1.0, 1.4, 1.76, 1.34]
```

```python
# Filtrar elementos
altos = [x for x in inf if x > 0.6]   # [0.7, 0.88, 0.67]

# Aplicar uma função a cada elemento
def taxar(x):
    return x * 1.1

taxado = [taxar(x) for x in inf]
```

O equivalente, sem list comprehension, ao exemplo de filtro:

```python
altos = []
for x in inf:
    if x > 0.6:
        altos.append(x)
```

12.2. Quando (não) usar

List comprehension é ideal para transformações simples de uma linha. Quando a lógica interna fica complexa (múltiplas condições, vários passos), um `for` tradicional costuma ser mais legível — lembre do princípio "legibilidade importa" do Capítulo 1. Regra prática: se a comprehension não cabe confortavelmente numa linha, prefira o `for` explícito.

12.3. Exemplo Resolvido — Achatando um dicionário aninhado

Enunciado: reaproveite o `pib_por_ano` do Capítulo 8.4 (dicionário de dicionários). Use list comprehension **aninhada** para transformá-lo em uma única lista de tuplas `(país, ano, valor)` — uma operação comum ao preparar dados aninhados para virarem uma tabela (o que a Apostila 2 fará automaticamente com pandas).

```python
pib_por_ano = {
    "Brasil": {"2023": 2.9, "2024": 3.1},
    "Chile":  {"2023": 0.2, "2024": 2.6},
}
```

Solução comentada:

```python
tabela = [
    (pais, ano, valor)
    for pais, series in pib_por_ano.items()
    for ano, valor in series.items()
]

print(tabela)
# [('Brasil', '2023', 2.9), ('Brasil', '2024', 3.1),
#  ('Chile', '2023', 0.2), ('Chile', '2024', 2.6)]
```

```
[('Brasil', '2023', 2.9), ('Brasil', '2024', 3.1), ('Chile', '2023', 0.2), ('Chile',
'2024', 2.6)]
```

Uma comprehension pode ter **mais de um `for`**, executados na mesma ordem em que apareceriam se fossem loops aninhados escritos à mão:

```python
tabela = []
for pais, series in pib_por_ano.items():
    for ano, valor in series.items():
        tabela.append((pais, ano, valor))
```

As duas versões produzem exatamente o mesmo resultado. A versão em comprehension é mais compacta; a versão com `for` explícito costuma ser mais fácil de depurar quando a lógica cresce (por exemplo, se fosse necessário um `if` adicional dentro do loop interno).

Exemplo resolvido — dict/set comprehension, expressões geradoras e `any`/`all`
A sintaxe de compreensão vale para *dicionários* e *conjuntos*. Uma **expressão geradora** (parênteses em vez de colchetes) não materializa a lista, e funções como `sum`, `any` e `all` a consomem aos poucos:

```
import sys
ipca = {"jan": 0.42, "fev": 0.83, "mar": 0.56, "abr": 0.44}
em_pontos_base = {mes: round(v * 100) for mes, v in ipca.items()}        # dict
comprehension
meses_altos = {mes for mes, v in ipca.items() if v > 0.5}               # set comprehension
print(em_pontos_base, meses_altos)
print("Algum mês acima de 0,8?", any(v > 0.8 for v in ipca.values()), "| todos positivos?",
all(v > 0 for v in ipca.values()))

lista_grande = [n * n for n in range(100_000)]
gerador_grande = (n * n for n in range(100_000))
print("Gerador ocupa menos memória que a lista:", sys.getsizeof(gerador_grande) <
sys.getsizeof(lista_grande))
print("Soma consumindo o gerador:", sum(n * n for n in range(1000)))
```

```
{'jan': 42, 'fev': 83, 'mar': 56, 'abr': 44} {'mar', 'fev'}
Algum mês acima de 0,8? True | todos positivos? True
Gerador ocupa menos memória que a lista: True
Soma consumindo o gerador: 332833500
```

Atenção — erro comum
Compreensões aninhadas com mais de dois níveis, ou com efeitos colaterais (`print`, alterar variáveis), ficam ilegíveis. Se a linha precisar de uma releitura, vire um laço comum: **clareza vence concisão**.

Capítulo 13 — Erros e depuração

13.1. Lendo uma mensagem de erro

Quando algo dá errado, Python interrompe a execução e mostra um **traceback**: o caminho que o programa percorreu até o erro, terminando na causa. A regra mais importante: **leia a última linha primeiro** — ela contém o tipo do erro e a mensagem; as linhas acima mostram em qual arquivo e linha do seu código o problema ocorreu.

```
# NameError — variável não definida
print(valor_inexistente)
# NameError: name 'valor_inexistente' is not defined

# TypeError — operação com tipo errado
"IPCA: " + 0.16
# TypeError: can only concatenate str (not "float") to str
"IPCA: " + str(0.16)  # correto

# IndexError — índice fora do intervalo da lista
lista = [1, 2]
lista[5]
# IndexError: list index out of range
```

```
# KeyError — chave que não existe no dicionário
d = {"a": 1}
d["b"]
# KeyError: 'b'

# ZeroDivisionError — divisão por zero
10 / 0
# ZeroDivisionError: division by zero
```

13.2. Tratando erros com try/except

Às vezes um erro é **esperado** (por exemplo, um dado que pode ou não existir) e você não quer que ele pare todo o script. Para isso existe `try/except`:

```
def taxa_segura(dados, chave):
    try:
        return dados[chave]
    except KeyError:
        print(f"Aviso: '{chave}' não encontrado, usando 0.0")
        return 0.0

pib = {"Brasil": 2.2}
print(taxa_segura(pib, "Brasil"))    # 2.2
print(taxa_segura(pib, "Uruguai"))   # Aviso: ... / 0.0
```

```
2.2
Aviso: 'Uruguai' não encontrado, usando 0.0
0.0
```

`try/except` não deve virar hábito para "esconder" erros de programação (isso mascara bugs reais) — o uso correto é para situações genuinamente esperadas, como dados ausentes numa fonte externa. Sobre esse assunto — validação e tratamento de dados reais — voltaremos com mais profundidade na Apostila 2.

13.3. Exemplo Resolvido — Tratando dois tipos de erro diferentes

Enunciado: escreva uma função `carrega_taxa(dados, chave)` que busca uma taxa num dicionário e a converte para `float`. Ela deve lidar com **dois** problemas possíveis: a chave pode não existir (`KeyError`), ou o valor pode ser um texto não numérico como `"N/D"` (`ValueError` ao converter). Em ambos os casos, retorne `None` — mas imprima uma mensagem diferente para cada causa.

```
dados = {"Brasil": "4.5", "Chile": "N/D"}
```

Solução comentada:

```
def carrega_taxa(dados, chave):
    try:
        return float(dados[chave])
    except KeyError:
        print(f"Aviso: '{chave}' não encontrado na base")
        return None
    except ValueError:
        print(f"Aviso: valor de '{chave}' não é numérico ({dados[chave]!r})")
        return None

dados = {"Brasil": "4.5", "Chile": "N/D"}
print(carrega_taxa(dados, "Brasil"))    # 4.5
print(carrega_taxa(dados, "Chile"))     # Aviso: valor não é numérico / None
print(carrega_taxa(dados, "Peru"))      # Aviso: não encontrado / None
```

```
4.5
Aviso: valor de 'Chile' não é numérico ('N/D')
None
Aviso: 'Peru' não encontrado na base
None
```

Um `try` pode ter **múltiplos `except`**, cada um capturando um tipo de erro diferente — Python testa cada `except` na ordem em que aparece e executa o primeiro que corresponder ao erro ocorrido. Isso permite dar um diagnóstico específico para cada causa possível de falha, em vez de um `except` genérico que trataria "chave ausente" e "valor inválido" da mesma forma, escondendo qual dos dois problemas realmente aconteceu.

Aprofundando a teoria — hierarquia de exceções, `finally`, exceções próprias e log
As exceções formam uma **hierarquia**: `except ArithmeticError` captura `ZeroDivisionError`, porque este é um tipo de `ArithmeticError`. Capturar `Exception` indiscriminadamente (ou, pior, um `except:` vazio) engole erros de programação junto com os esperados. O bloco `finally` roda *sempre*, com ou sem erro (fechar arquivos, liberar recursos). Para sinalizar erros do *seu domínio*, defina uma exceção própria e use `raise`. Para **diagnóstico**, prefira o módulo `logging` a `print`: níveis, horário e destino configuráveis.

```python
print("Hierarquia de ZeroDivisionError:", " -> ".join(c.__name__ for c in
ZeroDivisionError.__mro__))

class TaxaInvalida(ValueError):
    """Erro do domínio: uma taxa fora do intervalo plausível."""

def converter_percentual(texto: str) -> float:
    """Converte '4,5%' ou ' 4.5 % ' em 0.045; levanta TaxaInvalida se o texto não for uma
taxa plausível."""
    limpo = texto.strip().replace("%", "").replace(",", ".").strip()
    try:
        valor = float(limpo) / 100
    except ValueError:
        raise TaxaInvalida(f"não é número: {texto!r}") from None
    if not -0.5 <= valor <= 5:
        raise TaxaInvalida(f"fora do intervalo plausível: {valor:.1%}")
    return valor

for entrada in ["4,5%", " 10 % ", "N/D", "900%"]:
    try:
        print(f"{entrada!r:>9} -> {converter_percentual(entrada):.4f}")
    except TaxaInvalida as erro:
        print(f"{entrada!r:>9} -> TaxaInvalida: {erro}")
    finally:
        pass                                            # o 'finally' sempre roda (aqui só
ilustra a estrutura)
```

```
Hierarquia de ZeroDivisionError: ZeroDivisionError -> ArithmeticError -> Exception ->
BaseException -> object
   '4,5%' -> 0.0450
 ' 10 % ' -> 0.1000
    'N/D' -> TaxaInvalida: não é número: 'N/D'
   '900%' -> TaxaInvalida: fora do intervalo plausível: 900.0%
```

Capítulo 14 — Boas práticas de código

Um script curto de 10 linhas não exige disciplina; um projeto que você vai reabrir em três meses, sim. Alguns hábitos, baratos de adotar desde já, evitam retrabalho:

- **Siga PEP 8**: `snake_case` para variáveis e funções, 4 espaços de indentação (nunca tab), linhas de até ~80-100 caracteres.
- **Nomeie por significado, não por tipo**: `inflacao_mensal`, não `lista1` ou `x2`.
- **DRY (Don't Repeat Yourself)**: se você copiou e colou um bloco de código três vezes mudando só um número, isso deveria ser uma função com um parâmetro.
- **Comente o "porquê", não o "o quê"**: `# ajuste porque o BCB muda a metodologia do IPCA em 2020` é útil; `# soma 1 a x` (quando o código já diz isso) não é.
- **Funções pequenas e com um único propósito**: se o nome da função tem "e" no meio (`calcula_e_salva_e_plota`), considere dividir em três funções.

Tema (PEP 8)	Regra
Nomes	`snake_case` para variáveis e funções; `PascalCase` para classes; `MAIÚSCULAS` para constantes
Espaços	Espaço em torno de operadores (`a + b`) e depois de vírgula; sem espaço antes de `(`
Linhas	Até 79 a 100 caracteres; quebre dentro de parênteses
Imports	No topo do arquivo, um por linha: biblioteca padrão, depois terceiros, depois o seu código
Docstrings	Toda função pública descreve o que faz, os parâmetros e o que devolve
Comentários	Expliquem o *porquê* atualizem-se junto com o código

Exemplo resolvido — uma função bem escrita
Compare a versão "que funciona" com a versão "que dá para manter":

```
# versão difícil de manter
def f(a, b, c):
    return a*(1+b)**c

# versão legível: nome que diz o que faz, tipos, docstring e validação
def valor_futuro(valor_presente: float, taxa: float, periodos: int) -> float:
    """Valor futuro com juros compostos: VP * (1 + taxa) ** periodos.

    Args:
        valor_presente: capital inicial (> 0).
        taxa: taxa por período, em decimal (0,01 = 1%).
        periodos: número de períodos (inteiro >= 0).
    """
    if valor_presente <= 0 or periodos < 0:
        raise ValueError("valor_presente deve ser positivo e periodos, não negativo")
    return valor_presente * (1 + taxa) ** periodos

print(f"{valor_futuro(2500, 0.015, 8):.2f}")
```

```
2816.23
```

Intuição
Testar é executar a sua intuição. A forma mais simples de teste são *assertivas*: afirmações que quebram o programa se falsas. Um teste bom cobre o caso comum, o caso de borda e o caso de erro.

```python
assert valor_futuro(1000, 0.10, 1) == 1100.0000000000002 or abs(valor_futuro(1000, 0.10, 1)
- 1100) < 1e-9   # caso comum
assert valor_futuro(1000, 0.10, 0) == 1000
# borda: zero períodos
try:
    valor_futuro(-5, 0.1, 1)
# caso de erro
except ValueError:
    print("Os três testes passaram: caso comum, borda e erro.")
```

```
Os três testes passaram: caso comum, borda e erro.
```

Capítulo 15 — Exercícios para Executar (na mão)

Estes exercícios não exigem computador: o objetivo é treinar a leitura e o raciocínio sobre o código antes de escrevê-lo. Resolva no papel. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Trace de execução
Sem rodar no computador, escreva o que cada `print` abaixo exibiria, na ordem:

```python
x = 10
y = 3
print(x // y)
print(x % y)
print(x / y)
x, y = y, x
print(x, y)
```

Exercício 2 — Juros compostos na mão
Calcule, sem usar código, o valor futuro de um capital de R$ 500,00 aplicado a uma taxa de 2% ao mês, por 3 meses, usando a fórmula $\text{VF} = \text{VP} \cdot (1 + i)^n$. Mostre o cálculo mês a mês.

Exercício 3 — Avaliação lógica
Classifique cada expressão abaixo como `True` ou `False`, sem rodar no computador:

```python
a) 5 > 3 and 2 > 4
b) 5 > 3 or 2 > 4
c) not (5 > 3)
d) 0 and (10 / 0)     # cuidado com curto-circuito!
e) "" or "Brasil"
f) [] and [1, 2]
```

Exercício 4 — Encontre o erro
Cada trecho abaixo tem um erro. Sem rodar, diga qual é o tipo de erro (`NameError`, `TypeError`, `IndexError`, `KeyError` ou `ZeroDivisionError`) e por quê.

```python
a) preco = "10" + 5
b) paises = ["Brasil", "Chile"]
   print(paises[2])
c) pib = {"Brasil": 2.2}
   print(pib["Argentina"])
d) print(taxa_cambio)
e) resultado = 100 / (5 - 5)
```

Exercício 5 — Pseudocódigo de decisão
Descreva, em português estruturado (pseudocódigo, sem precisar ser Python válido), a lógica para classificar o regime de política monetária: se a Selic estiver acima da meta de inflação mais 5 pontos percentuais, é "contracionista"; se estiver entre a meta e a meta + 5, é "neutra"; caso contrário, é "expansionista". Depois, traduza seu pseudocódigo para Python.

Capítulo 16 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor. **As soluções não estão neste documento** — o objetivo é você rodar de verdade e ver o resultado; quando terminar, peça para eu revisar seu código.

Exercício 1 — Inflação acumulada
Crie uma função `inflacao_acumulada` que recebe uma lista de valores mensais (em %) e calcula a inflação acumulada no período, compondo cada taxa (não somando).

Exercício 2 — Maior e menor PIB
Dado um dicionário com PIB de países, retorne o país com maior PIB e o com menor.

```
pibs = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1, "Peru": 2.5, "México": 1.9}
```

Exercício 3 — Simulação de juros compostos mês a mês
Simule juros compostos: comece com R$ 1000, taxa de 1% ao mês, por 12 meses. Mostre o saldo a cada mês.

Exercício 4 — Valores acima da média
Crie uma função que recebe uma lista de números e retorna uma nova lista apenas com valores acima da média.

Exercício 5 — Média geométrica do IPCA
A inflação acumulada de N meses equivale a compor as taxas mensais; a taxa mensal **média** equivalente não é a média aritmética, é a média geométrica. Crie uma função `ipca_medio_geometrico(taxas)` que recebe uma lista de taxas mensais (%) e retorna a taxa mensal média que, composta N vezes, reproduz a mesma inflação acumulada.

Exercício 6 — Meses em recessão
Dada uma lista com a variação percentual do PIB trimestre a trimestre, conte quantos trimestres consecutivos, no pior trecho, o país ficou em recessão técnica (2 trimestres seguidos de PIB negativo já configura recessão técnica — mas aqui queremos o maior número de trimestres negativos consecutivos, mesmo que ultrapasse 2).

```
pib_trimestral = [0.5, -0.2, -0.3, -0.1, 0.8, -0.5, 1.0]
```

Exercício 7 — Crescimento a partir de dicionário aninhado
Usando o dicionário aninhado abaixo, crie uma função que recebe o dicionário, um país e dois anos, e retorna a variação percentual do PIB entre esses dois anos (não a variação ano a ano — a mudança acumulada entre os dois pontos).

```
pib_por_ano = {
    "Brasil": {"2022": 3.0, "2023": 2.9, "2024": 3.1, "2025": 2.2},
    "Chile":  {"2022": 2.4, "2023": 0.2, "2024": 2.6, "2025": 3.1},
}
```

Exercício 8 — Entrada validada com try/except
Crie uma função `taxa_para_float` que recebe uma string representando uma taxa (ex: `"4.5"`) e retorna o valor como `float`. Se a conversão falhar (ex: texto não numérico, como `"N/D"`), a função deve capturar o erro e retornar `None` em vez de quebrar o programa.

Exercício 9 — Classificador de cenário macroeconômico
Combine condicionais e funções: crie `classifica_cenario(pib, inflacao, desemprego)` que retorna uma string descrevendo o cenário, usando as regras: PIB negativo e inflação alta (>5%) → `"estagflação"`; PIB positivo e inflação baixa (<3%) e desemprego baixo (<8%) → `"expansão saudável"`; qualquer outro caso → `"cenário misto"`.

Exercício 10 — Séries "lado a lado" com list comprehension
Dadas duas listas de mesma extensão (IPCA e SELIC mês a mês), use list comprehension para criar uma terceira lista com o juro real aproximado (SELIC − IPCA) de cada mês, e depois outra lista comprehension para filtrar apenas os meses em que o juro real foi negativo.

```
selic = [10.5, 10.5, 10.75, 10.75, 11.0]
ipca =  [4.2, 4.5, 4.8, 5.1, 11.5]
```

Exercício 11 — Limpando dados com taxas inválidas
Reaproveite a função `taxa_para_float` do Exercício 8. Dada a lista abaixo, crie uma função `limpar_taxas` que devolva apenas os valores que puderam ser convertidos (descartando os inválidos), sem quebrar o programa.

```
taxas_brutas = ["4.5", "3.2", "N/D", "5.1", "", "2.8"]
```

Exercício 12 — Fechamentos de trimestre
Dada uma lista com 24 meses (índice 0 = janeiro do ano 1), use `enumerate` e o operador módulo para retornar apenas os **números dos meses** (1 a 24) que representam fechamento de trimestre (múltiplos de 3).

Exercício 13 — Parseando um rótulo de período
Strings como `"2024-Q1: 3.2%"` aparecem com frequência em bases de dados exportadas de forma "suja". Crie uma função `parse_periodo` que recebe essa string e retorna uma tupla `(ano, trimestre, valor)`, com `ano` e `trimestre` como `str` e `valor` como `float`.

Exercício 14 — Semestres via fatiamento
Dada uma série de 12 valores mensais de IPCA, use *slicing* (sem loop) para retornar duas listas: os 6 primeiros meses e os 6 últimos.

Exercício 15 — Cópia segura de uma base
Escreva uma função `atualiza_pib_seguro(base, pais, novo_valor)` que recebe um dicionário `{pais: pib}`, retorna uma **cópia nova** com o valor de `pais` atualizado, e garante que o dicionário original passado como argumento não seja alterado (evite o aliasing visto no Capítulo 7/8).

Exercício 16 — Contador de cenários
Reaproveite `classifica_cenario` do Exercício 9. Dada uma lista de tuplas `(pib, inflacao, desemprego)`, uma por trimestre, use um dicionário como **contador** para descobrir quantas vezes cada cenário ocorreu ao longo da série.

Exercício 17 — Estatísticas sem bibliotecas
Implemente `media_e_desvio(dados)` que retorna a média e o desvio padrão populacional de uma lista, **sem usar `statistics` nem `numpy`** — apenas o que foi visto neste capítulo (loops, list comprehension, `sum`, `len`, `** 0.5`).

$$\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n}}$$

Exercício 18 — Meses até dobrar (busca com while)
Generalize o exemplo do Capítulo 10: escreva `meses_para_dobrar(taxa_mensal)` que recebe uma taxa em decimal (ex: `0.01` para 1%) e retorna quantos meses são necessários para um capital dobrar de valor, usando `while`.

Exercício 19 — Acesso seguro em dicionário aninhado
Reaproveite `pib_por_ano` do Exercício 7. Escreva `pib_seguro(dados, pais, ano)` que retorna o valor se país e ano existirem, ou `None` — tratando tanto o caso de país inexistente quanto o de ano inexistente, com **um único bloco `try/except`** (dica: o erro é o mesmo tipo nos dois casos).

Exercício 20 — Projeto integrador: painel mensal
Este exercício combina praticamente tudo do capítulo. Dada a lista de dicionários abaixo (um por mês), escreva um programa que: (1) calcule o juro real de cada mês (`selic - ipca`); (2) identifique o mês com maior IPCA; (3) conte quantos meses tiveram juro real negativo; (4) imprima um resumo formatado.

```
painel = [
    {"mes": "jan", "selic": 10.5, "ipca": 0.42},
    {"mes": "fev", "selic": 10.5, "ipca": 0.83},
    {"mes": "mar", "selic": 10.75, "ipca": 0.56},
    {"mes": "abr", "selic": 10.75, "ipca": 0.44},
    {"mes": "mai", "selic": 11.0, "ipca": 1.20},
]
```

Capítulo 17 — Resumo

Conceito	Sintaxe
Print	`print(f"texto {var}")`
Variável (rótulo, não caixa)	`x = 10`
Lista (mutável)	`lista = [1, 2, 3]`
Dicionário (hash, acesso por chave)	`d = {"chave": valor}`
If/elif/else	`if x > 0:` / `elif` / `else:`
Loop for (iteráveis)	`for item in lista:`
Loop while (condição)	`while condicao:`
Função	`def nome(x): return x`
List comprehension	`[x*2 for x in lista if x > 0]`
Try/except	`try: ... except TipoDeErro: ...`
Comentário	`# isso é comentário`

Capítulo 18 — Mini-projeto Resolvido: Lendo e Analisando uma Série do BCB

Este mini-projeto usa os **dados reais** guardados neste repositório (o IPCA acumulado em 12 meses, baixado do Banco Central, em `ipca_12m.json`) para integrar o que a apostila ensinou: ler um arquivo, converter tipos, usar dicionários e listas, funções, tratamento de erro e formatação. O formato do arquivo é típico de APIs: uma lista de dicionários com a data (`"dd/mm/aaaa"`) e o valor como **texto**.

18.1. Ler e converter

```
import json

def ler_serie(caminho: str) -> dict:
    """Lê uma série do BCB (lista de {"data", "valor"}) e devolve {(ano, mes):
valor_float}."""
```

```
    with open(caminho, encoding="utf-8") as arquivo:
        registros = json.load(arquivo)
    serie = {}
    for registro in registros:
        dia, mes, ano = registro["data"].split("/")
        serie[(int(ano), int(mes))] = float(registro["valor"])
    return serie

ipca12 = ler_serie("ipca_12m.json")
primeiro, ultimo = min(ipca12), max(ipca12)
print(f"{len(ipca12)} observações | de {primeiro[1]:02d}/{primeiro[0]} a {ultimo[1]:02d}/
{ultimo[0]}")
print(f"Último valor: {ipca12[ultimo]:.2f}% em {ultimo[1]:02d}/{ultimo[0]}")
```

```
43 observações | de 01/2023 a 07/2026
Último valor: 4.44% em 07/2026
```

Intuição
Repare nas **escolhas de estrutura**: a chave do dicionário é uma **tupla** `(ano, mes)` (imutável, logo hashável), o valor é um **float** (convertido do texto), e a data virou algo ordenável. Cada decisão aproveita um conceito dos capítulos anteriores.

18.2. Analisar

```
def resumo_serie(serie: dict, teto: float = 4.5) -> dict:
    valores = list(serie.values())
    acima = [chave for chave, v in serie.items() if v > teto]
    return {
        "media": sum(valores) / len(valores),
        "maximo": max(serie.items(), key=lambda par: par[1]),
        "minimo": min(serie.items(), key=lambda par: par[1]),
        "meses_acima_do_teto": acima,
    }

r = resumo_serie(ipca12)
print(f"Média: {r['media']:.2f}%")
print(f"Máximo: {r['maximo'][1]:.2f}% em {r['maximo'][0][1]:02d}/{r['maximo'][0][0]}")
print(f"Mínimo: {r['minimo'][1]:.2f}% em {r['minimo'][0][1]:02d}/{r['minimo'][0][0]}")
print(f"Meses acima do teto da banda (4,5%): {len(r['meses_acima_do_teto'])} de
{len(ipca12)}")
```

```
Média: 4.61%
Máximo: 5.77% em 01/2023
Mínimo: 3.16% em 06/2023
Meses acima do teto da banda (4,5%): 24 de 43
```

18.3. Relatório formatado e tratamento de erro

```
def relatorio_por_ano(serie: dict) -> None:
    por_ano = {}
    for (ano, mes), valor in sorted(serie.items()):
        por_ano.setdefault(ano, []).append(valor)          # agrupa valores por ano
    print(f"{'Ano':<6}{'Média':>8}{'Máx':>8}{'Mín':>8}{'N':>4}")
    print("-" * 34)
    for ano, vals in por_ano.items():
        print(f"{ano:<6}{sum(vals) / len(vals):>8.2f}{max(vals):>8.2f}{min(vals):>8.2f}
{len(vals):>4}")
```

```python
relatorio_por_ano(ipca12)

try:
    ler_serie("arquivo_que_nao_existe.json")
except FileNotFoundError as erro:
    print(f"\nErro tratado com clareza: {erro.strerror}: {erro.filename!r}")
```

```
Ano      Média     Máx     Mín   N
-----------------------------------
2023      4.60    5.77    3.16  12
2024      4.37    4.87    3.69  12
2025      5.02    5.53    4.26  12
2026      4.37    4.72    3.81   7

Erro tratado com clareza: No such file or directory: 'arquivo_que_nao_existe.json'
```

Para ir além (sem solução aqui): acrescente um parâmetro para filtrar um intervalo de anos, e uma função que devolva o **mês em que a inflação mais variou** em relação ao mês anterior. Na Apostila 2, o mesmo arquivo vira um `DataFrame` e cada passo deste capítulo vira uma linha de pandas.

Capítulo 19 — Erros Comuns e Teste Rápido

19.1. Os erros mais comuns de quem está começando

#	Erro	Como evitar
1	Usar = onde queria ==	= atribui, == compara
2	Comparar floats com ==	`math.isclose` ou diferença com tolerância
3	Achar que `b = a` copia uma lista	Use `list(a)`, `a.copy()` ou `copy.deepcopy`
4	`lista = lista.sort()`	`sort()` devolve `None`; use `sorted()` ou chame `sort()` sem atribuir
5	Argumento padrão mutável `def f(x=[])`	Use `None` e crie dentro da função
6	`int()` esperando arredondar	`int` trunca; use `round`
7	`taxa or padrao` tratando 0 como ausente	Compare com `is None`
8	`except:` genérico	Capture a exceção específica
9	Misturar tabs e espaços na indentação	4 espaços, sempre
10	Esquecer a conversão de tipo ao ler arquivos	Todo dado de texto precisa de `float()`/`int()` explícito

19.2. Teste rápido: verdadeiro ou falso (com gabarito)

1. **V ou F:** `0.1 + 0.2 == 0.3` é `True` em Python. **Falso.** Dá `False` por causa da representação binária.
2. **V ou F:** Atribuir uma lista a outra variável cria uma cópia independente. **Falso.** Cria outro rótulo para o mesmo objeto.
3. **V ou F:** Strings são mutáveis. **Falso.** São imutáveis; `texto[0] = "x"` dá `TypeError`.
4. **V ou F:** `-7 // 2` é `-4`. **Verdadeiro.** A divisão inteira arredonda para menos infinito.
5. **V ou F:** Uma lista pode ser chave de dicionário. **Falso.** Não é hashável (mutável); use tupla.
6. **V ou F:** Um erro de sintaxe na linha 50 permite que as linhas 1 a 49 executem. **Falso.** A análise sintática falha antes de qualquer execução.
7. **V ou F:** `sorted(lista)` altera a lista original. **Falso.** Devolve uma nova lista.
8. **V ou F:** Em `[]` e `""`, `if x:` é falso. **Verdadeiro.** Coleções e strings vazias são falsas.
9. **V ou F:** Um gerador guarda todos os seus valores em memória. **Falso.** Produz um por vez, sob demanda.
10. **V ou F:** `round(2.5)` é `3`. **Falso.** É `2`: o arredondamento é "bancário" (para o par).

Próxima apostila

Quando terminar os exercícios acima, siga para a **Apostila 2 — pandas para Economia**. Lá você vai aprender a abrir tabelas, filtrar dados, agrupar, juntar bases e trabalhar com datas — 90% do que um economista faz no dia a dia. Os dicionários aninhados do Capítulo 8 vão reaparecer ali, agora como `DataFrame`.